

**Master in Data Science**

**Master Thesis**

**Neuro-Symbolic Predictive  
Process Monitoring with  
Procedural Background  
Knowledge**

Candidate: Zorzi Matteo

Supervisor: Ph.d. Donadello Ivan

Co-supervisor: Ph.d. Prof. Maggi Fabrizio Maria

07/10/2026

# Abstract

Predictive process monitoring forecasts how a running case will continue. Neuro-symbolic frameworks provide the model with knowledge about the process in two forms. A declarative model states the rules a trace must satisfy. A procedural model states the order a trace may follow. The frameworks proposed so far work with the declarative form. The most recent of them turns it into a differentiable penalty on the network's output. It reports that compliance increases with no loss in similarity. That result was obtained on filtered logs, with noise bounded at forty percent. Whether the gain comes from the knowledge or from the channel that carries it has not been tested.

This thesis brings the procedural form into the same setting. The local and global logic losses of the state of the art receive a mined Petri net. The local one judges a single step, while the global one judges a whole generated suffix. They are compared with five other models on identical knowledge, trunk, and protocol. Two models read the net's reachability automaton as a simpler penalty: one projected onto activity pairs and the other indexed by the state the prefix reaches. Three models receive the same net as an input feature: the marking, the marking on the graph of the net, and the whole marking history. The eight models are measured on four event logs at nine noise levels, up to eighty percent. The tasks are next-activity and suffix prediction under two protocols. The two protocols share every step and differ only in the partition from which the knowledge is mined: the test one or the training one. Every cell is repeated over ten seeds for 5760 runs.

Both logic losses reduce forbidden probability mass on every log. The global one gains similarity and accuracy at the same time. The local one holds that mass flat against noise. Beyond sixty percent corruption, it loses similarity on three logs and accuracy on two. The original noise range could not have reached that point. The feature channel, given the same net, produces no gain. Computational time separates the channels more sharply than accuracy. The cheapest variant is the most effective. The richest encoding nearly doubles the training budget for nothing. Where the knowledge is injected determines whether it has an effect at all.

# Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                   | <b>ii</b> |
| <b>Acronyms and Notation</b>                                      | <b>v</b>  |
| <b>1 Introduction</b>                                             | <b>1</b>  |
| 1.1 Context and Problem . . . . .                                 | 1         |
| 1.2 Gap . . . . .                                                 | 2         |
| 1.3 Research Questions . . . . .                                  | 2         |
| 1.4 Contributions . . . . .                                       | 3         |
| 1.5 Outline . . . . .                                             | 3         |
| <b>2 Background</b>                                               | <b>4</b>  |
| 2.1 Process Mining and Predictive Process Monitoring . . . . .    | 4         |
| 2.2 Event Logs and Prefix Logs . . . . .                          | 4         |
| 2.3 The Two Prediction Tasks . . . . .                            | 5         |
| 2.4 Background Knowledge: Petri Nets and Automata . . . . .       | 5         |
| 2.4.1 Petri Nets, Token Replay and Markings . . . . .             | 6         |
| 2.4.2 The Reachability Automaton of a Net . . . . .               | 6         |
| 2.5 Neural Networks Building Blocks . . . . .                     | 7         |
| 2.5.1 The Recurrent Cell . . . . .                                | 7         |
| 2.5.2 Graph Neural Networks . . . . .                             | 7         |
| 2.5.3 Training Objective and Differentiable Constraints . . . . . | 8         |
| 2.5.4 Scoring a Whole Sequence . . . . .                          | 8         |
| <b>3 Preliminary Work</b>                                         | <b>10</b> |
| 3.1 Knowledge as a Feature . . . . .                              | 10        |
| 3.2 Knowledge as a Loss . . . . .                                 | 10        |
| 3.3 Thesis Contribution . . . . .                                 | 11        |
| <b>4 Method</b>                                                   | <b>12</b> |
| 4.1 The Eight Models . . . . .                                    | 12        |
| 4.2 From Event Log to Prefix Log . . . . .                        | 14        |
| 4.3 Mining the Background Knowledge . . . . .                     | 14        |
| 4.4 Injecting Noise . . . . .                                     | 16        |
| 4.5 From a Prefix to a Symbolic State . . . . .                   | 16        |
| 4.6 The Shared Trunk . . . . .                                    | 17        |
| 4.6.1 Generating a Suffix . . . . .                               | 18        |
| 4.7 The Loss Channel . . . . .                                    | 18        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 4.7.1    | Penalizing Forbidden Mass . . . . .                                   | 18        |
| 4.7.2    | The Two Logic Losses of the State of the Art . . . . .                | 20        |
| 4.8      | The Feature Channel . . . . .                                         | 21        |
| 4.8.1    | The Flat Marking . . . . .                                            | 22        |
| 4.8.2    | The Heterogeneous Graph Encoder . . . . .                             | 22        |
| 4.8.3    | The Marking Sequence . . . . .                                        | 23        |
| 4.9      | Training and Experiments . . . . .                                    | 24        |
| <b>5</b> | <b>Experiments</b>                                                    | <b>25</b> |
| 5.1      | Setup . . . . .                                                       | 25        |
| 5.1.1    | Data . . . . .                                                        | 25        |
| 5.1.2    | Evaluation Protocols . . . . .                                        | 26        |
| 5.2      | Metrics . . . . .                                                     | 27        |
| 5.3      | Results . . . . .                                                     | 28        |
| 5.3.1    | Overview . . . . .                                                    | 28        |
| 5.3.2    | Next-Activity Accuracy . . . . .                                      | 30        |
| 5.3.3    | Suffix Violation Rate . . . . .                                       | 31        |
| 5.3.4    | Suffix Similarity . . . . .                                           | 32        |
| 5.3.5    | Forbidden Probability Mass . . . . .                                  | 33        |
| 5.3.6    | Computational Time . . . . .                                          | 34        |
| 5.4      | Answers to the Research Questions . . . . .                           | 36        |
| 5.4.1    | RQ1: The Same Knowledge as a Feature Achieves None of It . . . . .    | 36        |
| 5.4.2    | RQ2: The Form of the Automaton Matters, Its Memory Does Not . . . . . | 37        |
| 5.4.3    | RQ3: What Works Is Also What Takes Least Time . . . . .               | 37        |
| <b>6</b> | <b>Conclusion</b>                                                     | <b>38</b> |
| 6.1      | Answers to the Research Questions . . . . .                           | 38        |
| 6.2      | Contribution of the Two Protocols . . . . .                           | 39        |
| 6.3      | Limitations . . . . .                                                 | 39        |
| 6.4      | Future Work . . . . .                                                 | 39        |

# Acronyms and Notation

|                                  |                                                  |                          |                                                      |
|----------------------------------|--------------------------------------------------|--------------------------|------------------------------------------------------|
| <i>Acronyms</i>                  |                                                  |                          |                                                      |
| BPIC                             | Business Process Intelligence Challenge          | LTL                      | linear temporal logic                                |
| CSV                              | comma-separated values                           | $LTL_f$                  | LTL over finite traces                               |
| DFA                              | deterministic finite automaton                   | PM                       | process mining                                       |
| DL                               | Damerau-Levenshtein                              | PPM                      | predictive process monitoring                        |
| GLL                              | global logic loss                                | R-GCN                    | relational graph convolutional network               |
| GNN                              | graph neural network                             | RNN                      | recurrent neural network                             |
| GRU                              | gated recurrent unit                             | TACO                     | recurrent graph convolutional predictor              |
| LLL                              | local logic loss                                 | T-LEAF                   | temporal-logic embedded automata framework           |
| LSTM                             | long short-term memory                           |                          |                                                      |
| <i>Logs, traces and prefixes</i> |                                                  |                          |                                                      |
| $\mathcal{A}$                    | activity alphabet                                | $\sigma_{\leq k}$        | prefix of $\sigma$ of length $k$                     |
| $\mathcal{T}$                    | $\mathcal{A}$ with <PAD> and <START>             | $\sigma_{>k}$            | suffix of $\sigma$ , of length $n - k$               |
| $\sigma$                         | trace, a sequence over $\mathcal{A}$             | $n$                      | length of a trace                                    |
| $a_i$                            | activity at position $i$ of a trace              | $k$                      | cut point, $0 \leq k < n$                            |
| $\hat{a}_i$                      | activity predicted for the $i$ -th prefix        | $u, v$                   | two sequences compared by DL                         |
| $K$                              | partition the knowledge is mined from            | $\varepsilon$            | the empty prefix                                     |
| $C$                              | cases of a log                                   | $C_{tr}, C_{va}, C_{te}$ | training, validation and test cases                  |
| <i>Models and training</i>       |                                                  |                          |                                                      |
| $E$                              | embedding matrix over $\mathcal{T}$              | $\mathcal{L}$            | training objective                                   |
| $h$                              | final recurrent hidden state                     | $\Omega$                 | logic term of the objective                          |
| $W$                              | linear classifier applied to $h$                 | $\lambda$                | weight of the logic term                             |
| $z, p$                           | logits and distribution over $\mathcal{A}$       | $N$                      | prefixes evaluated                                   |
| $B$                              | prefixes in a batch                              | $L$                      | padded length of a batch                             |
| $d_{out}$                        | width of an encoder's output                     | $R$                      | sequence of encoder readouts                         |
| $t_i$                            | last real token of the $i$ -th prefix            | $y_i$                    | target of the $i$ -th example                        |
| $\ell$                           | median trace length of the test partition        | $\hat{p}$                | acceptance of a rollout                              |
| $r_i$                            | rejected mass of the $i$ -th example             | $\alpha$                 | mixture of supervision and logic                     |
| $H$                              | rollout horizon                                  | $w_i$                    | gate on the supervised term                          |
| $S$                              | rollout samples                                  | $\tau$                   | Gumbel-Softmax temperature                           |
| $g$                              | noise of the Gumbel-Softmax                      | $u_j$                    | uniform draw on $(0, 1)$                             |
| $\parallel$                      | concatenation of two vectors                     | $\mathbb{1}[\cdot]$      | indicator, 1 when the condition holds                |
| acc                              | next-activity accuracy                           | sim                      | Damerau-Levenshtein similarity                       |
| CE                               | cross-entropy                                    | forbidden                | forbidden probability mass                           |
| <i>Petri nets</i>                |                                                  |                          |                                                      |
| $P, T$                           | places and transitions of a net                  | $H_P^i, H_T$             | place and transition states                          |
| $\bullet t, t \bullet$           | pre-set and post-set of $t$                      | $A_{PT}, A_{TP}$         | adjacency matrices                                   |
| $m \in \mathbb{N}^{ P }$         | marking, a vector of token counts                | $\phi$                   | linear map followed by a ReLU                        |
| $H^i$                            | node states after $i$ updates                    | $A$                      | adjacency matrix of a graph                          |
| <i>Automata and masks</i>        |                                                  |                          |                                                      |
| $Q, q$                           | states of the reachability automaton             | $M^{net}$                | projected mask, $ \mathcal{T}  \times  \mathcal{A} $ |
| $\delta$                         | transition function of an automaton              | $M^{state}$              | state-indexed mask, $ Q  \times  \mathcal{A} $       |
| allowed( $q$ )                   | activities permitted in state $q$                | $q_0, F$                 | initial and accepting states                         |
| $\Delta, \mu, \eta$              | transition tensor, initial and accepting vectors | $\hat{q}_t$              | relaxed automaton state                              |
|                                  |                                                  | $\tilde{x}, \tilde{o}_t$ | Gumbel sample and acceptance score                   |

# Chapter 1

## Introduction

A business process is a set of activities carried out to reach a goal, such as handling an order or treating a patient. Information systems record the execution of these processes. Process mining analyzes these records [1], and predictive process monitoring forecasts the future of a case while it is still running [2, 3]. The input consists of the events of the case observed so far. Outputs can be the next activity, remaining sequence, estimated completion time, or final outcome. This thesis studies the first two prediction tasks.

Neuro-symbolic frameworks are emerging in process mining. They give the model access to knowledge about the process itself. That knowledge is temporal, since it constrains the order in which activities may occur. It takes two forms. A declarative model states the rules a trace must satisfy [4]. A procedural model states the orderings a trace may follow [5]. The frameworks proposed so far work with the declarative form [6, 7]. This thesis brings the procedural form into the same setting and compares the two.

### 1.1 Context and Problem

A *case* is one execution of the process, and an *event* is the record of one activity inside a case. Most predictive process monitoring systems read a case's initial sequence of events [2]. This sequence is called a prefix, and a recurrent network learns to predict what follows it [8]. This approach ignores the knowledge that process mining extracts from the log. Running a discovery algorithm on the log produces a directly-follows graph, a Petri net, or a set of declarative constraints [4]. These constructs state possible future events, required activity order, and the case's current state. A predictive process monitoring model receives none of them. The network must rediscover this information from the activity labels alone. Two families of methods address this gap. The *feature channel* supplies the network with an input feature extracted from the process model, before classification [9, 10]. The *loss channel* adds to the cross-entropy loss a differentiable penalty on the probability mass assigned to forbidden behavior [11, 12, 7].

The two families handle background knowledge differently. They share no log, baseline, architecture, or metric, and both report gains. No published gain is attributable to the channel since no study varies the channel and holds the rest fixed.

## 1.2 Gap

The state of the art introduced by Mezini [6, 7] defines the current integration of the loss channel. Two penalties are derived from an automaton compiled from mined Declare constraints. The cited work reports that the compliance of predictions grows without compromising similarity to the ground truth. Furthermore, the advantage is reported to grow as noise increases within the training log.

The reported results are obtained under favorable conditions, and noise reaches only forty percent. Every trace that violates the mined specification is removed from both partitions, and for this reason, the baseline starts near full compliance. The background knowledge used is a Declare model, and it is not established that the penalties survive when the knowledge is instead a Petri net discovered by a miner. This thesis supplies the same procedural knowledge, mined from the same log partition, to the same recurrent model through each available channel. Three new encodings of increasing richness, together with two simpler penalties, are compared against the state-of-the-art penalties. All seven share the same knowledge in different forms, and every variant is then run at noise levels reaching eighty percent of the events in the log. No trace is removed from either partition, and every variant is scored against the same automaton. The channel is therefore the only factor that varies, and any difference between variants is attributable to it.

## 1.3 Research Questions

The thesis is organized around three questions.

**RQ1** Does the *same* knowledge injected as an input *feature* achieve what it achieves as a *loss*?

Knowledge is held fixed in these experiments, and only the channel changes. An automaton derived from a Petri net and the marking of that net are two different views of the same process model. Since the knowledge does not change, any difference that appears is a difference between the channels.

**RQ2** Among the losses, does the form of the process model specification matter?

The same automaton can be projected onto pairs of activities or kept whole and indexed by the state a prefix reaches. This changes the representation of the automaton and also the information it carries. The projection discards memory, and whether that memory is worth keeping is measured.

**RQ3** What does each channel cost in computational time?

A channel can leave conformance and predictive accuracy unchanged at the same computational time as the baseline, or at twice that time. The implementation of different techniques needs to take account of the training budget. A small improvement at double the time may not justify the additional computational time.

## 1.4 Contributions

The state of the art proposes two logic losses, which are provided with the same knowledge as every other variant: the reachability automaton of a mined Petri net. These losses are compared against a baseline model, three models that receive the marking of a Petri net as a feature at three levels of richness, and two models trained with penalties derived from the reachability automaton of the same Petri net. Seven of the eight carry the same background knowledge in different forms, while the baseline carries none. Two experimental protocols are tested, differing only in the partition from which the knowledge is mined. The conditions under which the two state-of-the-art losses hold are reported. Computational time is measured for every variant. None of the work reviewed in chapter 3 reports it. The channel that costs the least computational time is also the most effective one.

## 1.5 Outline

Chapter 2 defines vocabulary and notation. It covers event logs, prefix logs, prediction tasks, process models, knowledge sources, and neural blocks. Chapter 3 reviews two families of methods. Each is read against the research questions, so what it leaves untested is named. Chapter 4 describes the design. It defines the eight models, the four penalties incorporated with knowledge, the marking pipeline, and its three encoders. Chapter 5 reports the measurements. It states the two protocols as equals, defines the metrics, and answers the three research questions in order, one per subsection. Chapter 6 answers each question and reports what the two protocols together establish. It states limitations and names the experiments that should come next.

## Chapter 2

# Background

This chapter defines the theory and the notation that the thesis uses in chapters 4 and 5. The first part is symbolic: event logs, prefix logs, the two prediction tasks, and the procedural knowledge the thesis injects. The second part is neural and holds the building blocks of every model.

### 2.1 Process Mining and Predictive Process Monitoring

Information systems record the execution of business processes using event logs. PM uses these records to reconstruct how processes actually behave, rather than how they are assumed to behave. PM is organized into three families of techniques [1]. Discovery builds a process model from a log alone. Conformance checking tests a model against observed behavior. Enhancement uses the log to extend an existing model. This thesis uses the first two. PPM works on cases that are still running [13, 3]. Given a partial trace, the task is to predict its outcome or future activities.

### 2.2 Event Logs and Prefix Logs

An event log stores traces produced by the execution of the same process, and it is a multiset since two cases can produce the same trace [1]. A trace records one case as a sequence of events, and each event refers to an activity of the alphabet  $\mathcal{A}$ . An event may carry further attributes, such as the resource that performed it. Throughout this thesis, a trace is written  $\sigma = \langle a_1, \dots, a_n \rangle$ , and only the case identifier, the activity label and the timestamp are used.

At prediction time, predictive methods observe only partial traces. Given  $\sigma$  and a cut point  $0 \leq k < n$ , the *prefix*  $\sigma_{\leq k} = \langle a_1, \dots, a_k \rangle$  is the available history at prediction time, and the *suffix*  $\sigma_{> k} = \langle a_{k+1}, \dots, a_n \rangle$  is what needs to be predicted [8, 7]. A log needs to be converted into a *prefix log* before training [2, 8]. Every trace of length  $n$  produces  $n$  supervised examples. Table 2.1 cuts a three-trace log into its prefix log.

A  $\langle \text{START} \rangle$  symbol is inserted prior to every prefix, and the unused positions of a batch are filled with the  $\langle \text{PAD} \rangle$  symbol. Neither symbol is an activity. The alphabet that the model reads from,  $\mathcal{T} = \mathcal{A} \cup \{\langle \text{START} \rangle, \langle \text{PAD} \rangle\}$ , is larger than the alphabet it predicts over,  $\mathcal{A}$ .

| Trace                        | $n$ | $k$ | Prefix $\sigma_{\leq k}$  | Next Activity $a_{k+1}$ | Suffix $\sigma_{>k}$         |
|------------------------------|-----|-----|---------------------------|-------------------------|------------------------------|
| $\langle a, b, d \rangle$    | 3   | 0   | $\langle \rangle$         | $a$                     | $\langle a, b, d \rangle$    |
|                              |     | 1   | $\langle a \rangle$       | $b$                     | $\langle b, d \rangle$       |
|                              |     | 2   | $\langle a, b \rangle$    | $d$                     | $\langle d \rangle$          |
| $\langle a, c, d \rangle$    | 3   | 0   | $\langle \rangle$         | $a$                     | $\langle a, c, d \rangle$    |
|                              |     | 1   | $\langle a \rangle$       | $c$                     | $\langle c, d \rangle$       |
|                              |     | 2   | $\langle a, c \rangle$    | $d$                     | $\langle d \rangle$          |
| $\langle a, b, b, d \rangle$ | 4   | 0   | $\langle \rangle$         | $a$                     | $\langle a, b, b, d \rangle$ |
|                              |     | 1   | $\langle a \rangle$       | $b$                     | $\langle b, b, d \rangle$    |
|                              |     | 2   | $\langle a, b \rangle$    | $b$                     | $\langle b, d \rangle$       |
|                              |     | 3   | $\langle a, b, b \rangle$ | $d$                     | $\langle d \rangle$          |

Table 2.1: The prefix log of  $\{\langle a, b, d \rangle, \langle a, c, d \rangle, \langle a, b, b, d \rangle\}$ , the log used as a running example throughout this chapter. The prefix  $\langle a \rangle$  occurs three times and carries two different targets.

## 2.3 The Two Prediction Tasks

A predictive model is trained on the prefix log of a training log to predict what follows a prefix. The prefix log defines the two tasks studied in this thesis. Both read the same prefix  $\sigma_{\leq k}$  and differ only in what they return.

Next-activity prediction maps a prefix  $\sigma_{\leq k}$  to the single next activity  $a_{k+1}$ . The prediction is a classification task over the alphabet  $\mathcal{A}$  [8].

Suffix generation maps  $\sigma_{\leq k}$  to the entirety of  $\sigma_{>k}$  and is produced autoregressively [8, 7]. The model ingests its own predictions at each step in order to produce a complete trace. Errors compound since a wrong prediction is fed back and conditions every step after it. No end-of-trace symbol is used, and the generation stops after a fixed number of steps defined by the suffix length.

The model does not read the same input at training time and at generation time. At training time, every step is conditioned on the real prefix, and the model never sees its own generated tokens. At generation time, only the first step is a next-activity prediction, whereas every step thereafter is conditioned on the model's own output.

## 2.4 Background Knowledge: Petri Nets and Automata

The background knowledge used in this thesis is procedural. These artifacts state which orderings of activities are allowed, and not which rules a trace must satisfy. Every artifact used in the models derives from a single Petri net discovered from the training or test partition of the log. The net stores the history of an execution in a marking. The marking can be given as a feature to a model directly, or the reachability automaton can be derived from the net. The automaton turns that marking into a constraint on the next activity. That constraint can be indexed by the state the execution has reached or projected onto the last activity of the prefix.

The state of the art uses a Declare model. This model is a set of  $LTL_f$  rules over the activities, compiled into a deterministic finite automaton [4, 14, 7]. Chapter 4 applies its two logic losses to the net above so that every variant carries the same knowledge.

### 2.4.1 Petri Nets, Token Replay and Markings

A *Petri net* is a bipartite directed graph over two disjoint sets of nodes: *places*  $P$  and *transitions*  $T$  [5, 15]. Transitions and places are connected using arcs that run only between nodes of different types. The pre-set  $\bullet t \subseteq P$  of a transition is the set of places with an arc into it. The post-set  $t \bullet$  is the set of places it has an arc into. A marking  $m \in \mathbb{N}^{|P|}$  defines how many tokens are at each place and represents the state of one execution. Transition  $t$  is *enabled* in marking  $m$  when every place of its pre-set holds a token. When a transition fires, it takes one token from each place of  $\bullet t$  and puts one token into each place of  $t \bullet$  [5]. The arcs are oriented, so consuming a token and producing one are different relations. *Silent* transitions carry no activity label and fire without any observed event, moving a token through an optional branch. Figure 2.1 shows a net over the running example.

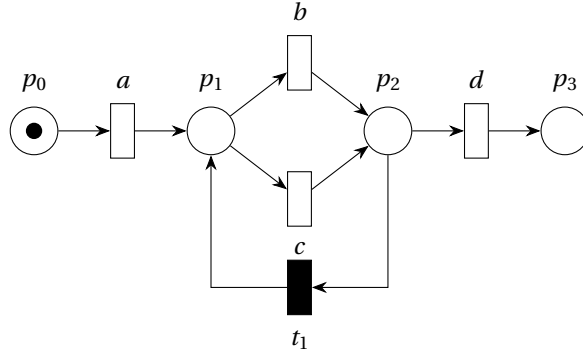


Figure 2.1: A net over the log of table 2.1. Circles are places, rectangles transitions, and the filled rectangle is the silent transition  $t_1$ , which is what allows  $b$  to repeat. The token in  $p_0$  is the initial marking  $m_0 = (1, 0, 0, 0)$ ; replaying  $\langle a, b \rangle$  moves it to  $p_2$ .

Petri nets are discovered from a log via a mining algorithm. This thesis uses the inductive miner with a noise threshold. It filters infrequent behavior and guarantees a sound net [16]. Silent transitions help achieve soundness. A prefix is mapped to a marking via *token replay*. The events of the prefix are fired in order [9].

### 2.4.2 The Reachability Automaton of a Net

A deterministic finite automaton is a tuple  $(Q, \mathcal{A}, \delta, q_0, F)$  [17].  $Q$  is the finite set of states,  $\delta : Q \times \mathcal{A} \rightarrow Q$  is the transition function,  $q_0 \in Q$  is the initial state, and  $F \subseteq Q$  is the set of accepting states. A trace is accepted when reading it from  $q_0$  ends in a state of  $F$ . A Petri net does not directly specify which activities may follow a prefix, but its *reachability automaton* does. It is a deterministic finite automaton whose states are markings and whose edges carry the activities of the visible transitions. It is built by traversing every marking reachable from the initial one, and it recognizes exactly the traces the net can perform [5, 15]. Figure 2.2 shows the automaton of the net of fig. 2.1.

The automaton is built from a discovered net, and three properties of that construction follow. Silent transitions make the marking graph nondeterministic, so a state is not a single marking but the set of markings reachable through zero or more silent firings [17]. Activities outside the net's alphabet self-loop on every state, so the automaton never prohibits them. Every activity that a state cannot take leads to a trap state, and rejected traces end there.

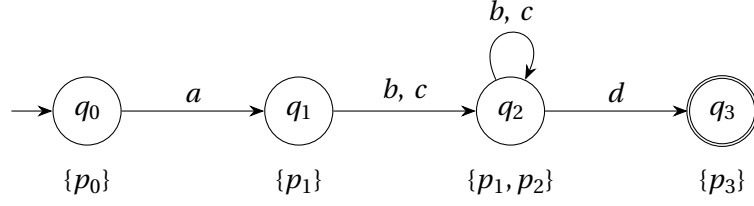


Figure 2.2: The reachability automaton of the net of fig. 2.1. State  $q_2$  holds two markings, because  $t_1$  returns a token to  $p_1$  without consuming an event.

The state of the automaton is the marking and not the last activity, so it separates contexts that an activity cannot separate.

## 2.5 Neural Networks Building Blocks

A neural network is a model whose parameters are learned from examples. A neural sequence model is a neural network whose input is a sequence, such as a prefix. In this thesis, it is a recurrent cell trained against an objective. A cell can read a graph as well as a sequence, and an objective can carry a term in addition to the error on the label.

### 2.5.1 The Recurrent Cell

Each prefix activity is mapped to a vector by the embedding  $E$ . A recurrent network RNN reads that sequence, and the linear classifier  $W$  predicts the next activity from its final state [8]. Figure 2.3 shows it.

$$h = \text{RNN}(E[\sigma_{\leq k}]), \quad z = Wh, \quad p = \text{softmax}(z), \quad (2.1)$$

where  $h$  is the state after the prefix's last activity, and  $p$  is a distribution over  $\mathcal{A}$ .

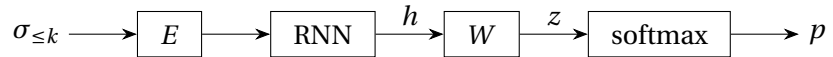


Figure 2.3: Standard architecture for next-activity prediction: the embedding  $E$ , the recurrent cell, the linear classifier  $W$ , and the softmax over  $\mathcal{A}$ .

A simple recurrent cell overwrites its state at every step, and early information from long prefixes is lost. The LSTM keeps long-term memory in an additional cell state. The cell is composed of gates that learn how much of the old state to keep [18]. The task requires that retention because the presence of a constraint is not weakened by the events that follow it. PPM uses the LSTM for this reason [8].

### 2.5.2 Graph Neural Networks

A marking is a vector of token counts. Information about which places are connected is not included in that vector. A graph neural network reads the structure instead. GNNs share one mechanism. A node updates its state based on the states of its neighbors. Repeating the updates widens the set of neighbor nodes on which the node state depends [19, 20]. With

the node states stacked in a matrix  $H^i$ , one update is

$$H^{i+1} = \phi(A^\top H^i), \quad (2.2)$$

where  $A$  is the adjacency matrix of the graph and  $\phi$  is a map with learned weights. The product  $A^\top H^i$  sums, for every node, the states of the nodes with an arc into it. Figure 2.4 shows two updates on the net of fig. 2.1.

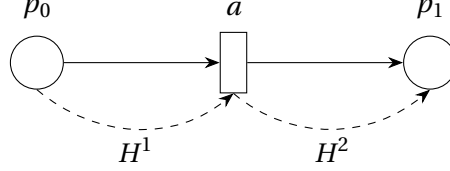


Figure 2.4: Two updates of eq. (2.2) on the first arcs of the net of fig. 2.1. After the first update the state of  $a$  depends on  $p_0$ , and after the second the state of  $p_1$  depends on  $p_0$  through  $a$ .

A *heterogeneous* network keeps separate weights per relation, where a relation is a type of arc [21].

### 2.5.3 Training Objective and Differentiable Constraints

The supervised term of the objective is the cross-entropy between  $p$ , the prediction, and the true next activity,

$$\text{CE}(p, a_{k+1}) = -\log p[a_{k+1}], \quad (2.3)$$

that is, the negative log-probability that  $p$  assigns to  $a_{k+1}$ . A network trained on that term alone assigns probabilities to activities that the process model does not allow.

Symbolic knowledge is put in a differentiable form and added as a second term, weighted against the first,

$$\mathcal{L} = \text{CE}(p, a_{k+1}) + \lambda \Omega(p), \quad (2.4)$$

where  $\Omega$  is the logic term, which measures the distance of the prediction from the constraint, and  $\lambda$  is its weight [11, 22, 7].

The penalty is soft. The constraint is satisfied when the activity to which  $p$  assigns the highest probability,  $\hat{a}_{k+1} = \arg \max_{c \in \mathcal{A}} p[c]$ , is a valid prediction. The direct penalty would be the number of invalid predictions. That count is a step function of the logits, so its gradient is zero. Probability moves continuously with the logits. A penalty on the probability of those same predictions has a gradient and can be used in training. The constraint is therefore taught through the loss.

### 2.5.4 Scoring a Whole Sequence

A penalty that reads a single predicted distribution scores one step. Some constraints can only be checked on a complete execution. A prefix that satisfies such a constraint may still be extended into a trace that violates it. Scoring a whole generated sequence requires a differentiable automaton [23], a differentiable sample [24], and a Monte Carlo estimate. All three are used in section 4.7.2.

The first is a differentiable automaton. It accepts distributions instead of symbols. A deterministic finite automaton with states  $Q$  and alphabet  $\mathcal{A}$  can be written as a transition

tensor  $\Delta \in \{0, 1\}^{|\mathcal{A}| \times |Q| \times |Q|}$ , an initial vector  $\mu$ , and an acceptance vector  $\eta$ . The tensor holds one  $|Q| \times |Q|$  matrix per activity. The two vectors have one entry per state, with  $\mu$  marking the initial state and  $\eta$  the accepting states. Written this way, a state update is a product,

$$\tilde{q}_0 = \mu, \quad \tilde{q}_t = \sum_{j=1}^{|\mathcal{A}|} x_t[j] \tilde{q}_{t-1} \Delta[j], \quad \tilde{o}_t = \langle \tilde{q}_t, \eta \rangle, \quad (2.5)$$

where  $x_t$  is a distribution over the alphabet. When  $x_t$  and  $\tilde{q}_{t-1}$  are one-hot, the result is one-hot and the product does exactly what the original transition does. The soft form does not distort the automaton; it only extends it to uncertain inputs. It is differentiable in both arguments. The acceptance score  $\tilde{o}_t$  replaces a binary answer with a value in  $[0, 1]$ . That is what makes a model checker usable as a gradient.

The second is a differentiable sample. A sequence is generated autoregressively, so the model feeds its own choice back as the next input. An argmax has no useful derivative. The Gumbel-Softmax reparameterization draws a nearly one-hot vector from a categorical distribution while remaining differentiable,

$$\tilde{x} = \text{softmax}\left(\frac{\log p + g}{\tau}\right), \quad g_j = -\log(-\log u_j), \quad u_j \sim \mathcal{U}(0, 1), \quad (2.6)$$

where  $g$  is the noise that turns the softmax into a sample, and the temperature  $\tau$  controls how close that sample is to one-hot. A large  $\tau$  flattens the sample toward uniformity. Feeding  $\tilde{x}$  back into the recurrence keeps the generated sequence differentiable.

The third is a Monte Carlo estimate, which replaces an exponential sum that cannot be computed. The probability that a trace drawn from the model satisfies a specification is a sum over every trace the vocabulary allows. An estimate can be drawn over  $S$  sequences by averaging their acceptance scores. It is differentiable because all its components are also differentiable. Each estimate costs  $S$  rollouts to the horizon, so this penalty is more expensive than one that reads a single distribution.

These are the components from which a neuro-symbolic model is built. Chapter 3 reviews the systems that have assembled them and what they leave untested.

## Chapter 3

# Preliminary Work

The literature injects process knowledge in two ways. The first injects the knowledge into an input feature of a sequence model. The second injects the knowledge into a term of the loss function. This chapter reviews both, shows what they leave untested, and states what this thesis contributes.

### 3.1 Knowledge as a Feature

The direct form of the feature channel is implemented by Theis and Darabi [9]. The method discovers a Petri net, replays each prefix, and feeds the resulting marking to the network as an extra input vector. It establishes that the marking is usable as a feature. The marking used is also the weakest form because the marking of the last event overwrites everything the case did before.

An improvement to the feature channel is introduced by TACO [10]. When a loop completes, the marking returns to a state it already occupied. Distinct prefixes become identical inputs. It names this defect *loop overwriting*. This problem is solved by keeping one marking per event and processing the whole trajectory. TACO's contribution is architectural. The linear maps inside a GRU are replaced by graph convolutions over the net. The hidden state is indexed by place, and the recurrence and the graph propagation are interleaved rather than stacked. TACO also reduces the bipartite net to a homogeneous *place graph* to save computational time.

A recent benchmark reviews the graph neural networks applied to next-activity prediction [25]. It reports a gain over traditional approaches on some logs. The approaches it reviews build the graph from the events of the prefix. Chiorrini et al. do the same, encoding one execution as an instance graph and enriching it with the attributes of its events [26]. The structure they read is the trace and not the process model.

These prediction methods compare a knowledge-carrying model against a knowledge-free baseline. None of them asks whether the same knowledge, delivered through a different channel, would have the same effect. This is what RQ1 asks.

### 3.2 Knowledge as a Loss

The loss channel starts with a different premise. The outputs of the network are read as predicates, and a symbolic constraint is a formula over them. That formula is compiled into

a differentiable penalty.

Two systems apply the pattern to sequence prediction. Differentiable temporal logic [11] expresses constraints as LTL formulas over the model’s per-frame predictions. It evaluates them with differentiable operators and adds the result to the cross entropy. T-LEAF [12] compiles an  $LTL_f$  specification into a deterministic finite automaton. It embeds that automaton with a graph network, so the model is scored against the whole object and not against a summary of it.

Neither of them is applied to event logs. Logic over event logs has been made differentiable for conformance checking, where  $LTL_f$  is given a fuzzy semantics and evaluated on uncertain data [27]. The system that reads event logs and predicts them is the work this thesis benchmarks.

Mezini [6, 7] introduces the loss channel into PPM. From the log, Declare constraints are mined, compiled through  $LTL_f$  into a deterministic finite automaton, and tensorized so that the automaton becomes differentiable. Two penalties are built on it. The *local* loss penalizes, at each step, the mass the model places on rejected activities. The *global* loss allows the model to generate a complete suffix and reads the final acceptance score. Both are stated in full in section 4.7.2. The results report that both improve the satisfiability of the generated traces without compromising similarity to the ground truth and that the advantage grows with the level of injected noise.

Three features of that evaluation limit the claim. The noise range stops at forty percent. Every trace that violates the mined specification is removed from *both* partitions before training. The baseline begins close to full compliance, and improvements are bounded by the construction of the experiment. The process model specification is a Declare model compiled through  $LTL_f$ . The penalties have been tested only with automata compiled from declarative models, and never with automata derived from a procedural model.

The two losses of section 4.7.2 are the state of the art against which this thesis measures itself. The reproduction is not a direct comparison to published values. The log subsets differ; the process model specification differs; the filtering step differs; the split ratio differs; and the results are published as figures rather than tables. This thesis reproduces the method and tests the claims.

Keeping the automaton whole costs more than projecting it onto pairs of activities, and nobody measures the two forms against each other. This is what RQ2 asks.

### 3.3 Thesis Contribution

This thesis makes two contributions.

The first, and more important, is the use of procedural knowledge. Few predictive models use a Petri net internally. This thesis discovers one net per log and injects it to the network in five forms. Two are new terms of the loss, derived from the reachability automaton of that net. Three are encodings of the same net as input features. The two logic losses in the state of the art have never been given a Petri net. This thesis is the first to do so.

The second is the measurement of computational time. Injection mechanisms are compared based on what they achieve and not on what they consume. One penalty reads a single distribution. Another rolls a batch through a differentiable automaton. They do not take the same time, and none of the works reports the difference. This is what RQ3 asks.

This thesis holds the background knowledge, the trunk, and the protocol fixed, and changes only the channel and the form of the knowledge. Chapter 4 describes that design.

# Chapter 4

## Method

This thesis compares different injection channels. Eight models share one recurrent trunk and one training procedure. Table 4.1 lists them, and the sections after section 4.1 describe each of them in full. Seven of them also share one process model specification. These models differ only in how that specification reaches the network, either as a term of the loss or as an input feature. The eighth is the baseline, which receives none and is the reference against which the others are measured. Holding the knowledge fixed and varying the channel serves as a comparison for the internal mechanisms rather than for the nets themselves.

This chapter follows the pipeline in the order in which it runs. An event log becomes a prefix log. The symbolic artifacts are mined from a designated partition of the same log, then the training data are corrupted, and every prefix acquires a symbolic state. Then the models are instantiated, and they have only the trunk in common.

Two of the loss variants, the local and global logic losses, are implemented by Mezini [6, 7]. They are included here as the state of the art for comparison, and they consider the same automaton that the others read.

### 4.1 The Eight Models

Table 4.1 lists the eight models in two blocks below the baseline. The first varies the *form of the penalty* while holding the architecture fixed; the second varies the *encoder* while holding the objective fixed.

The four loss variants have *the same architecture* as the baseline, from the parameter count to the initialization under a given seed. Only the objective differs. Any difference derives only from the different losses. The three encoders differ only in the type of information they provide. Both blocks receive the *same* Petri net.

The trunk remains the same in every variant and follows this sequence of operations: a prefix is embedded, read by an LSTM [18], and mapped by a linear classifier to a distribution. The embedding replaces each token of the prefix with a row of the matrix  $E$ , a vector of 32 values learned together with the rest of the network. The LSTM size is taken from the work of Mezini et al. [7]. In this way, the architecture remains the same, and any difference between two variants comes from the channel. No per-cell hyperparameter search is performed. Figure 4.1 shows the trunk and the eight variants defined in this chapter.

Every variant reads the same knowledge. The Petri net is discovered once per log and per protocol. The automaton that the four loss variants read is its reachability automaton. The benchmark, therefore, compares knowledge injection rather than process models.

| Model             | Description                                                                                              | What it isolates   |
|-------------------|----------------------------------------------------------------------------------------------------------|--------------------|
| Baseline          | The recurrent trunk trained on the cross-entropy alone, with no knowledge.                               | —                  |
| Projected net     | Penalizes the probability of activities the reachability automaton forbids after the last activity.      | the constraint     |
| State-indexed net | The same penalty, indexed by the state the prefix reaches in the automaton instead of its last activity. | its memory         |
| Local loss        | Penalizes the forbidden probability logarithmically and drops the supervision on rejected targets.       | the penalty's form |
| Global loss       | Generates a suffix through a differentiable automaton and penalizes a low acceptance at its end.         | the horizon        |
| Marking           | Gives the marking of the prefix to the classifier as a feature.                                          | the state          |
| Graph encoder     | Propagates the marking over the net structure with a heterogeneous graph network.                        | the structure      |
| Marking sequence  | Encodes one marking per event with the graph encoder and summarizes the history with a GRU.              | the history        |

Table 4.1: The eight models. All share the recurrent trunk of eq. (2.1) and, apart from the baseline, the same discovered Petri net.

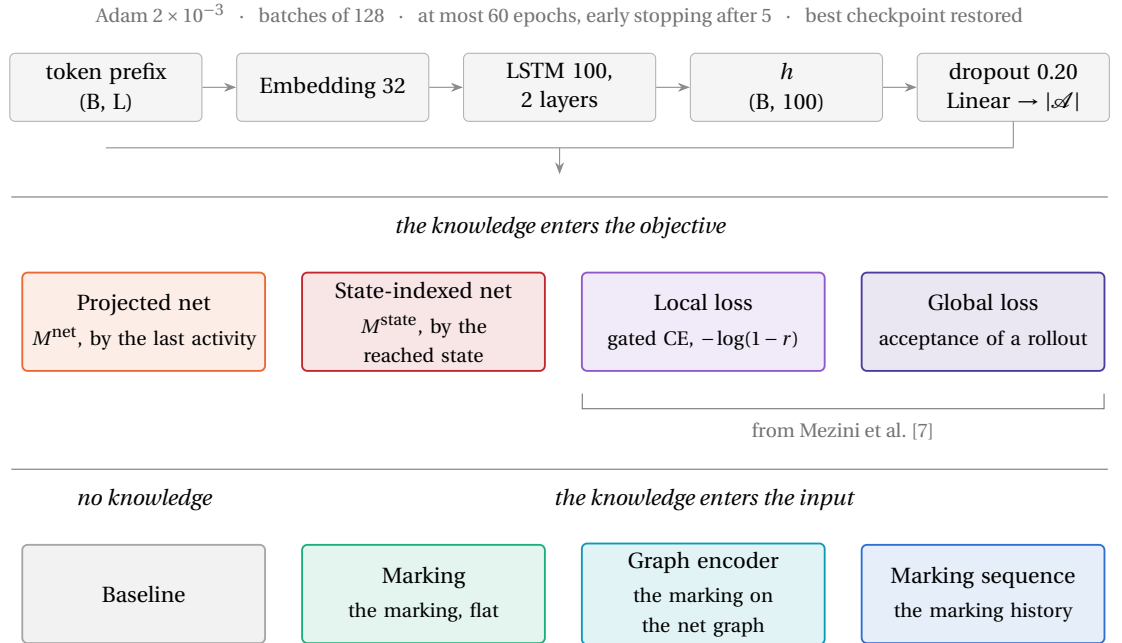


Figure 4.1: The eight models share the trunk drawn at the top; below they are grouped by knowledge injection: the objective, the input, or neither.

## 4.2 From Event Log to Prefix Log

Before generating any prefix, the log cases need to be assigned to a partition. The set of cases present in a log is  $C$ , and  $|C|$  is its size. A split divides that set into three partitions,

$$C = C_{\text{tr}} \sqcup C_{\text{va}} \sqcup C_{\text{te}}, \quad (4.1)$$

holding 70%, 15%, and 15% of the cases. Cases are ordered by the timestamp of their first event [7] and that order is cut by position,

$$C_{\text{tr}} = c_{1: \lfloor 0.70|C| \rfloor}, \quad C_{\text{va}} = c_{\lfloor 0.70|C| \rfloor + 1: \lfloor 0.85|C| \rfloor}, \quad C_{\text{te}} = c_{\lfloor 0.85|C| \rfloor + 1: |C|}, \quad (4.2)$$

so that the training cases precede the validation cases, which in turn precede the test cases. Ties are broken by position in the log. The split is therefore deterministic and the same for every seed.

A complete trace  $\sigma$  of length  $n$  expands into  $n$  nested prefixes  $\sigma_{\leq k}$ , of increasing length  $0 \leq k < n$ . The prefix log of a partition is a multiset since the same prefix may occur across cases with a different next activity. The size of a prefix log is the number of events in its partition. The input of an example is one token longer than the prefix it encodes because it begins with  $\langle \text{START} \rangle$ . Table 4.2 shows the expansion of one trace.

| $k$ | Input                              | Target |
|-----|------------------------------------|--------|
| 0   | $\langle \text{START} \rangle$     | $a$    |
| 1   | $\langle \text{START} \rangle a$   | $b$    |
| 2   | $\langle \text{START} \rangle ab$  | $c$    |
| 3   | $\langle \text{START} \rangle abc$ | $d$    |

Table 4.2: A trace of four events contributes four examples to a prefix log. The cut point starts at  $k = 0$ , so the first predicts the opening activity from an empty history.

The same expansion also defines the second task. The suffix of a prefix is everything that follows it in the trace, and the suffix task asks the model to generate that continuation from its own predictions.

Resources, lifecycle, and every case or event attribute are discarded. The objective of this study is procedural knowledge, and an attribute stream would give the models a second source of information.

The vocabulary  $\mathcal{A}$  of a log is the set of its activity labels, sorted so that each label has the same index. The vocabulary is built over all three partitions, so every case remains representable and none is dropped.

Prefix log examples have different lengths. A batch is padded using the  $\langle \text{PAD} \rangle$  token to the length of the longest member. The batch also carries an explicit vector of the true lengths. The lengths exclude the padding steps, so the final state of a sequence is the state after its last real token.

## 4.3 Mining the Background Knowledge

Two symbolic artifacts are mined from a single designated partition  $K$  of the event log. The partition is determined by the protocol, and mining occurs before any noise is injected. Figure 4.2 shows what is derived from each.

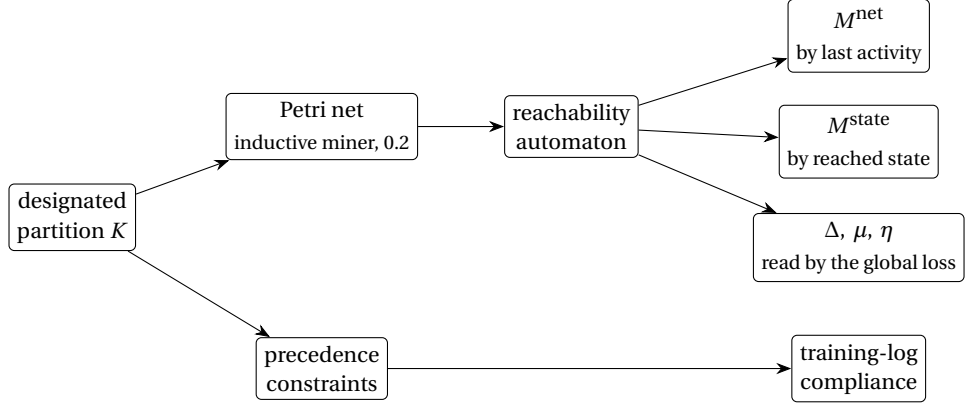


Figure 4.2: What is derived from the designated partition. The Petri net is the only source of constraint, and the three objects on the right are all read from its reachability automaton. The precedence constraints produce no mask and enter no model: they only measure how much the injected noise has damaged the training log (table 5.3).

The first artifact is a Petri net, discovered with the inductive miner at a noise threshold of 0.2, using the Python library pm4py [28]. Edges below the threshold are removed before the net is built. The result is a process tree [16], hence a sound net. Places and transitions are stored in a deterministic order, so that the index of a place in a marking is the same as its index in the adjacency matrices.

The net is the *only* source of constraint in this thesis, and everything else in this section is derived from it. Two forms of knowledge that come from different objects cannot be compared as channels, because any difference between them confounds the channel with the source. A *mask* is a boolean matrix whose entries state which activities the net allows next. Deriving every mask and every feature from one discovered net removes that problem.

The loss channel consumes an automaton derived from the net. The reachability automaton is built by exploring the markings the net can reach, so its states are markings and its transitions are the firings that connect them. The construction is that of section 2.4.2. Its alphabet is taken from the activity vocabulary. An activity pruned by the miner appears in no transition of the net. Building the automaton over the net’s labels alone would give it no edge, and a mask read from that automaton would forbid it everywhere. Keeping it in the alphabet gives it a self-loop on every state instead. The net then neither permits nor forbids it, and no conformance metric counts it as a violation.

The automaton yields two masks. Projected onto activity pairs, it gives  $M^{\text{net}}$ , indexed by the last activity of the prefix. Kept unprojected, it gives  $M^{\text{state}}$ , indexed by the state the prefix reaches, which retains the memory that the projection discards.

The second artifact mined from  $K$  is a set of precedence constraints. Each constraint states that  $b$  happens only after  $a$  [4]. Precedence is the only template used, because it captures the order between activities, which is what the noise injection breaks. Its support is the number of traces containing  $b$ , and its confidence is the fraction of those in which  $a$  occurs before the first  $b$  [29]. A candidate is retained when it holds with a confidence of at least 0.95 on a support of at least  $\max(3, |K|/10)$  traces, and at most fifteen constraints are kept. Scaling the support threshold with the size of that partition rather than with the size of the training set allows the same thresholds to be used when the knowledge comes from the smaller test block. These constraints enter no model, and their only role is to measure

how much the injected noise has damaged the training log, which table 5.3 reports.

## 4.4 Injecting Noise

On a clean log, there is little for a conformance term to correct. Noise is what makes the value of a constraint measurable. It is introduced into the training data to give a constraint something to recover from.

Event noise replaces the label of a fraction of the events in the training traces. The replacement activity is drawn from the remaining vocabulary, so every selected event actually changes. Events are sampled globally over the log rather than trace by trace. The requested fraction therefore holds over the whole log, and a single trace may keep all of its events or have several relabeled. Because the damaged object is the trace, the corruption propagates into every later prefix of the same case, into the marking the net replays for those prefixes [9, 10], and into whether a trace still satisfies the mined constraints. This is the noise model of Mezini et al. [7]. The reference work stops at forty percent. In this thesis, nine levels are used, from no noise to eighty percent in steps of ten.

Table 4.3 places the model on the trace of table 4.2.

| $k$ | Clean                       | Event noise                  |
|-----|-----------------------------|------------------------------|
| 0   | $\varepsilon \rightarrow a$ | $\varepsilon \rightarrow a$  |
| 1   | $a \rightarrow b$           | $a \rightarrow \mathbf{x}$   |
| 2   | $ab \rightarrow c$          | $a\mathbf{x} \rightarrow c$  |
| 3   | $abc \rightarrow d$         | $a\mathbf{x}c \rightarrow d$ |

Table 4.3: Event noise on the prefix log of Table 4.2. Each row is written as prefix  $\rightarrow$  target.  $\langle \text{START} \rangle$  is omitted, and the empty prefix is  $\varepsilon$ . One relabeled activity, in bold, reaches three of the four examples. The damaged object is the trace.

Three properties keep a comparison across noise levels readable. The fraction is taken over the events of the partition, so at a given level, the same proportion of labels changes on every log. The model neither inserts nor deletes an event, so the number of examples is constant across levels. Only the training partition is corrupted, so the validation and test sets remain clean, and every metric is measured against them.

## 4.5 From a Prefix to a Symbolic State

Each prefix is mapped to a marking by token replay. The replay is configured to walk through silent transitions and not to stop on unfitting behavior. The time it takes is determined by the size of the discovered net. The same prefix is replayed once for the marking encodings and once for the marking sequence, and prefixes repeat across cases. Markings are therefore cached on the prefix that produces them.

Two facts follow from this configuration. The empty prefix does not yield the initial marking. Every silent transition enabled at the start fires before the first event arrives. Unfitting prefixes still produce a marking because the replay inserts the tokens it lacks rather than stopping. A place can then hold more than one token. This is why markings are token counts and not sets of occupied places.

The marking sequence needs the whole marking history, not just the final marking. For a prefix of  $k$  events, it is a sequence of  $k + 1$  markings. The first is the marking of the empty prefix, so the sequence aligns step for step with the input stream, whose first element is  $\langle \text{START} \rangle$ . The last element of the sequence is the final marking. The marking sequence therefore contains everything the graph encoder receives, plus the history.

A prefix also carries the state that its replay reaches in the net’s reachability automaton. It is read only by the loss of eq. (4.7), which indexes its constraint on the automaton’s state instead of on the last activity.

## 4.6 The Shared Trunk

All eight models instantiate eq. (2.1) with the configuration of table 4.4, which collects the sizes of the trunk, the optimizer, and the schedule that each run follows.

| Component      | Value                                                           |
|----------------|-----------------------------------------------------------------|
| Embedding      | dimension 32                                                    |
| Recurrent cell | LSTM, two layers of 100 units                                   |
| Dropout [30]   | 0.20, between the two layers and before the classifier          |
| Optimizer      | Adam, learning rate $2 \times 10^{-3}$ , weight decay $10^{-5}$ |
| Batch          | 128 prefixes                                                    |
| Epochs         | at most sixty, at least five                                    |
| Early stopping | five epochs without an improvement in validation loss           |
| Checkpoint     | the best validation loss, restored before any metric            |

Table 4.4: The trunk and the schedule, shared by all eight models. The sizes are taken from the reference work [7].

The trunk is identical across the eight models, because a benchmark that varied it would measure a difference between architectures instead of a difference between channels.

The recurrent cell can be either an LSTM network [18] or a GRU [31]. This thesis uses only the LSTM. Padding is excluded from the recurrence by packing, and the padding token has a fixed zero embedding.

The trunk is written so that a marking encoder is an *injected module*. The constructor accepts an object satisfying the contract  $\text{forward}(\text{markings}, \text{lengths}) \rightarrow (B, d_{\text{out}})$ , together with its output dimension. The classifier then operates on the concatenation  $[h \parallel \text{enc}(m)]$ :

$$z = W[h \parallel \text{enc}(m)], \quad W \in \mathbb{R}^{|\mathcal{A}| \times (100 + d_{\text{out}})}. \quad (4.3)$$

The baseline and the four loss variants receive no encoder and  $d_{\text{out}} = 0$ . Switching from the flat marking to the graph encoder is a change of constructor argument, and there is no path by which any other difference can enter.

A batch can carry both symbolic streams: the final marking and the marking sequence. The consumption of the stream is decided by the encoder. That decision is not inferred from the batch, since the same batch can carry both.

#### 4.6.1 Generating a Suffix

The suffix task reuses the trained next-activity model. Given a prefix, the model predicts a distribution. The arg-max activity is appended to the prefix, and the extended sequence is reused for prediction. Decoding is greedy, and the automaton never overrides a choice, so a generated suffix can violate the net. Suffix conformance is therefore needed as a measurement. For the variants that read a marking, that marking must come from the model's *own* predictions during generation. Each new marking comes from replaying the whole generated prefix.

Generation stops when the produced activities reach the true suffix length, and no end-of-trace symbol is used. The length is given and not predicted by design. This separates the quality of the predicted activities from the quality of the predicted length, so differences between variants come from the activities alone. As a result, the Damerau-Levenshtein values reported in chapter 5 are higher than they would be if the model had to predict where to stop. They remain valid as comparisons between the eight variants.

### 4.7 The Loss Channel

Four models do not modify the architecture and add only a term to the objective. Their only difference is the form of that term. The trunk, the optimizer, the schedule, and the Petri net they draw on are identical. Two of the four are introduced by this thesis and are described first. The other two are the state of the art and are described in section 4.7.2.

#### 4.7.1 Penalizing Forbidden Mass

Two of the models introduced by this thesis that modify only the objective are the *projected net* and the *state-indexed net*. Both read the reachability automaton of section 2.4.2, and the only difference between them is how they index it. Figure 4.3 details the architecture of the two methods together with the baseline. The three share the same architecture, and the penalty is the only difference.

The projected net flattens the automaton onto the last token of the prefix, which gives a boolean *allowed mask*,

$$M^{\text{net}} \in \{0, 1\}^{|\mathcal{T}| \times |\mathcal{A}|}, \quad M^{\text{net}}[a, c] = 1 \iff \exists q' \in \delta(\cdot, a) : c \in \text{allowed}(q'). \quad (4.4)$$

After  $a$ , the mask allows whatever is permitted in any state that  $a$  can lead to. Rows are indexed by  $\mathcal{T}$  because special symbols such as <START> need to be taken into consideration. Columns are instead indexed by  $\mathcal{A}$  because no special symbol can be a prediction. An unconstrained token forbids nothing, so its row is set to all ones, permitting every activity.

The projection keeps only the last activity, so it forgets which state the prefix reached. An activity that leads to two different states therefore receives the union of what those states permit. The state-indexed net retains that state because its mask is indexed by the automaton's state instead of the last token:

$$M^{\text{state}} \in \{0, 1\}^{|Q| \times |\mathcal{A}|}, \quad M^{\text{state}}[q, c] = 1 \iff c \in \text{allowed}(q), \quad (4.5)$$

where  $q$  is the state the automaton reaches after replaying the prefix. The trap state and any state that permits nothing receive the all-ones row. Marking the trap as forbidden instead would not change training, because a constant mass has no gradient.

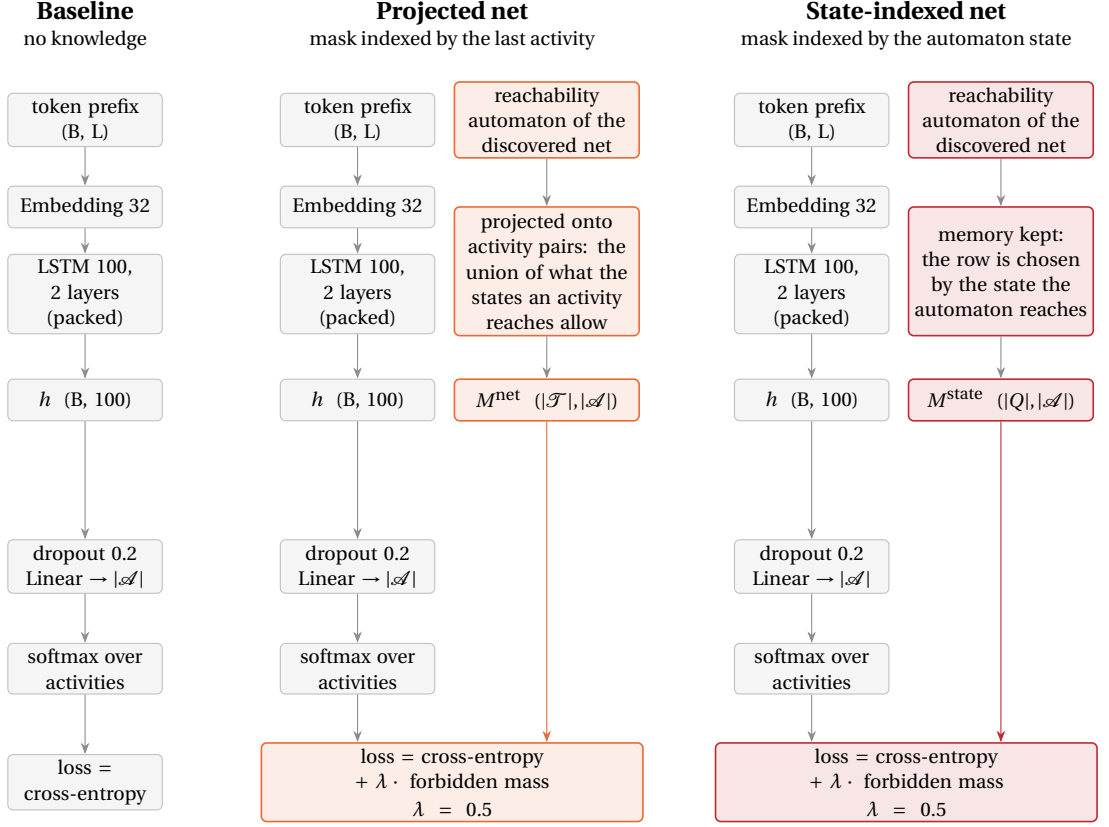


Figure 4.3: The baseline and the two net-derived losses. The neural skeleton is grey and identical in the three, and the two penalties differ only in the mask that feeds them.

Both add a differentiable penalty to the probability mass assigned to forbidden activities. With  $M^{\text{net}}$  the projected mask of eq. (4.4) and  $t_i$  the last real token of the  $i$ -th prefix, the objective is

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \frac{1}{B} \sum_{i=1}^B \sum_{c: M^{\text{net}}[t_i, c]=0} \text{softmax}(z_i)[c]. \quad (4.6)$$

The gradient of the second term pushes probability away from activities that the discovered net does not permit after the current one.

The state-indexed net replaces the projected mask with  $M^{\text{state}}$  of eq. (4.5), and the penalty becomes

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \frac{1}{B} \sum_{i=1}^B \sum_{c: M^{\text{state}}[q_i, c]=0} \text{softmax}(z_i)[c], \quad (4.7)$$

with  $q_i$  the state reached by replaying the  $i$ -th prefix. It is the only model in this thesis in which the net reaches the objective *with its memory* rather than flattened onto activity pairs. T-LEAF keeps the automaton whole rather than flattening it [12], and the state-indexed net implemented in this thesis follows that idea.

The mask enters only the objective and never the prediction, so the accuracy reported is the model's own. The penalty is linear in the forbidden mass, so its gradient is constant. The influence of the extra loss term is fixed at  $\lambda = 0.5$ , and no parameter search was performed during the experiments.

### 4.7.2 The Two Logic Losses of the State of the Art

The local and global logic losses are Mezini’s contribution [6, 7]. Figure 4.4 shows their architecture.

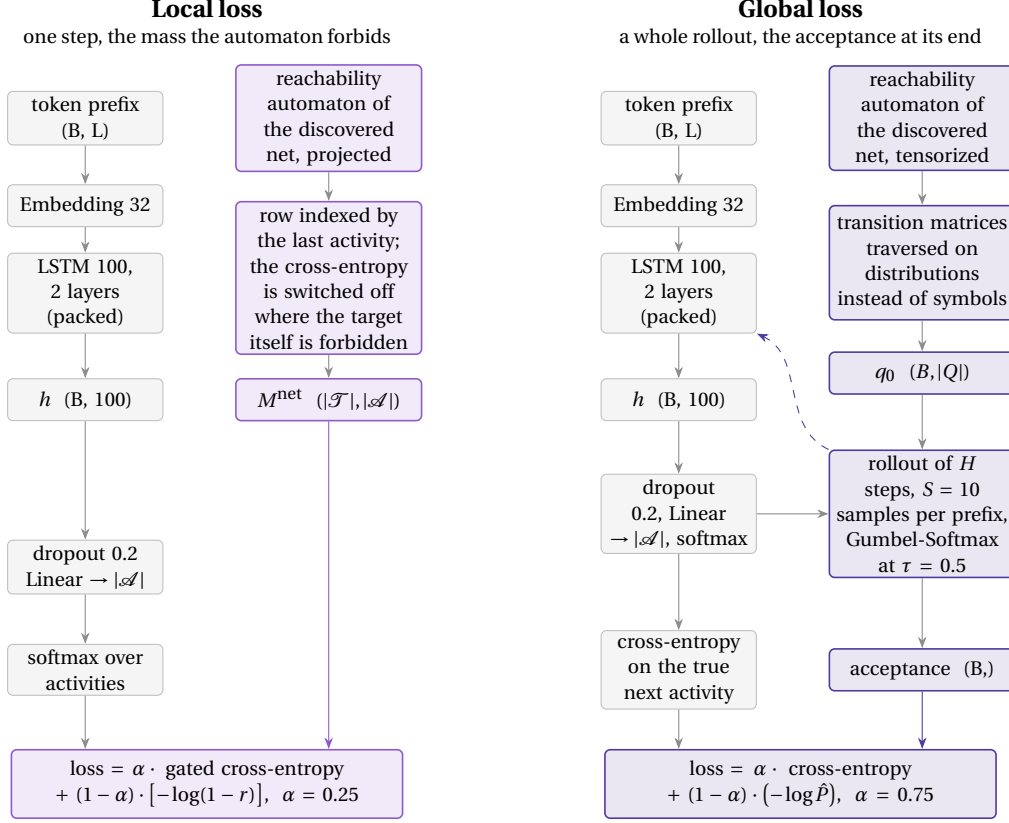


Figure 4.4: The two logic losses of Mezini et al. [7], on the same trunk and the same automaton as the rest of the benchmark. Only the objective changes.

The objective of the benchmark is to compare channels on identical knowledge. The *local loss* and the *global loss* are tested against five knowledge-carrying models, with the same Petri net knowledge as a background. The structure of both penalties is unchanged, as is the logic. Only three things change. The knowledge comes from the reachability automaton of the discovered net instead of the original DFA, compiled from a Declare model through  $\text{LTL}_f$ . The batch is a prefix with one target rather than a whole trace, so the local loss is computed once per example instead of at every step. The index spaces differ since the input tokens include  $\langle \text{START} \rangle$  and  $\langle \text{PAD} \rangle$ , while the output classes consist of activities alone.

**The local loss.** Let  $t_i$  be the last token of the  $i$ -th prefix, the same index that eq. (4.6) uses. The rejected mass of an example is the probability that the model places on activities that the projected mask forbids.

$$r_i = \sum_{c: M^{\text{net}}[t_i, c] = 0} \text{softmax}(z_i)[c], \quad (4.8)$$

and the penalty is its negative log-complement, averaged over the batch:

$$\mathcal{L}_{\text{loc}} = -\frac{1}{B} \sum_{i=1}^B \log(1 - r_i). \quad (4.9)$$

The supervised term is the cross-entropy over the examples that the automaton accepts. Rejected targets are left out, so the supervision is *gated*,

$$w_i = \mathbb{1}[M^{\text{net}}[t_i, y_i] = 1], \quad \mathcal{L}_{\text{CE}}^w = \frac{\sum_i w_i \cdot \text{CE}(z_i, y_i)}{\sum_i w_i}, \quad (4.10)$$

and the two are mixed by a single weight,

$$\mathcal{L}_{\text{LLL}} = \alpha \mathcal{L}_{\text{CE}}^w + (1 - \alpha) \mathcal{L}_{\text{loc}}, \quad \alpha = 0.25. \quad (4.11)$$

Under noise, the rejected targets are corrupted labels, and the gate prevents the model from learning them. Unlike the linear form of eq. (4.6), the penalty of eq. (4.9) grows without bound as the rejected mass approaches one.

**The global loss.** The global loss scores a whole predicted suffix instead of a single step. The automaton it reads is the projection of eq. (4.4), tensorized as in eq. (2.5), so its states are activities and not markings. From the state the prefix reaches, the model feeds its own predictions forward for  $H$  steps, sampling each one with a Gumbel-Softmax [24] at temperature  $\tau$ , as in eq. (2.6). The automaton consumes the sampled activities as they arrive. After the last step, an end symbol is consumed, and the remaining acceptance mass is evaluated,

$$\hat{P}_i = \frac{1}{S} \sum_{n=1}^S \langle \tilde{q}_{H+1}^{(i,n)}, \eta \rangle, \quad \mathcal{L}_{\text{glob}} = -\frac{1}{B} \sum_{i=1}^B \log \hat{P}_i. \quad (4.12)$$

The mixture with the supervised term is the same as in eq. (4.11),

$$\mathcal{L}_{\text{GLL}} = \alpha \mathcal{L}_{\text{CE}} + (1 - \alpha) \mathcal{L}_{\text{glob}}, \quad \alpha = 0.75. \quad (4.13)$$

Generation stops after  $H = \lfloor \ell/2 \rfloor + 2$  steps, with  $\ell$  as the median trace length of the test partition, and  $S = 10$  samples drawn at  $\tau = 0.5$ . Generating  $H$  steps costs more than scoring one.

The local penalty is a probability mass at one step, while the global penalty is the negative logarithm of an acceptance score over a whole generation. The two therefore need different  $\alpha$  values, which Mezini et al. search for [7], and this thesis declares in advance.

## 4.8 The Feature Channel

The second channel leaves the objective unchanged and gives the marking of section 4.5 to the classifier as an input. Three encoders read that marking, and each carries more information than the one before. Figure 4.5 shows their architecture.

The first gives the symbolic state, the second the structure of the net over that state, and the third the history.

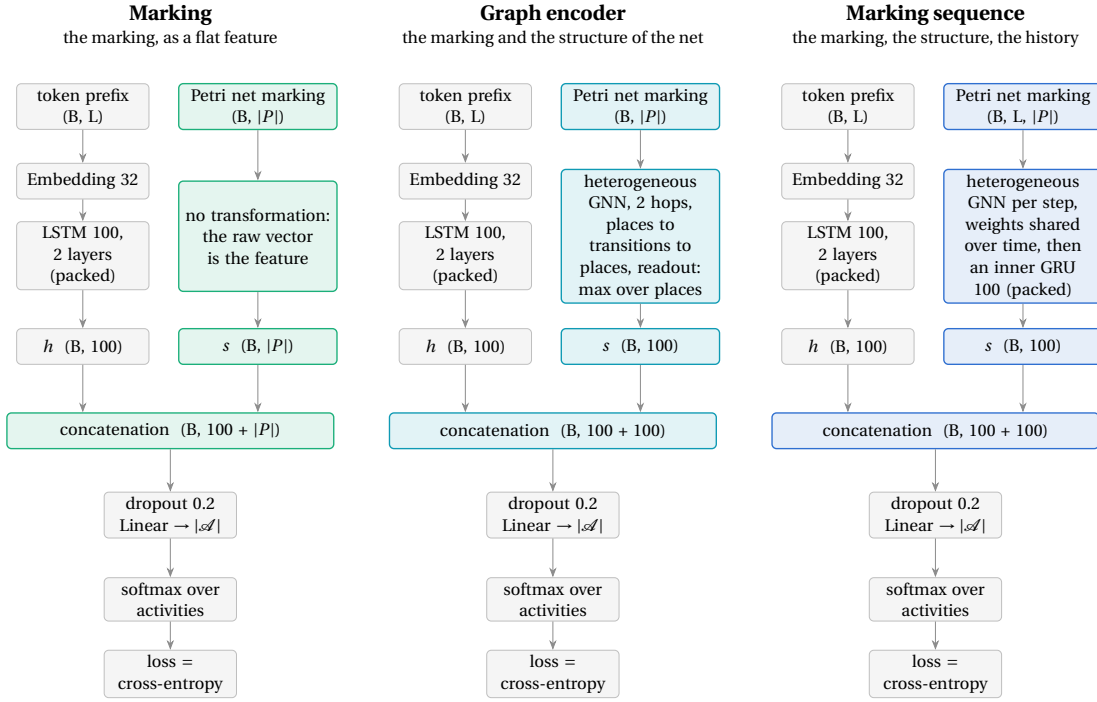


Figure 4.5: The three encoders of the feature channel. The neural skeleton is grey and identical in the three, and in colour is what each variant adds to it.

#### 4.8.1 The Flat Marking

The simplest encoder is the identity. The marking is cast to float and concatenated to the recurrent state,

$$\text{enc}(m) = m \in \mathbb{R}^{B \times |P|}, \quad d_{\text{out}} = |P|. \quad (4.14)$$

Each row of  $\text{enc}(m)$  is the marking of one prefix in the batch, its  $p$ -th entry is the number of tokens in place  $p$ , and  $d_{\text{out}} = |P|$  is the width concatenated to  $h$  in eq. (4.3). For the prefix  $\langle a, b \rangle$  on the net of fig. 2.1, the row is  $(0, 0, 1, 0)$ . The model receives the state of the process but no information about how the places are connected. This is the encoding of Theis and Darabi [9].

#### 4.8.2 The Heterogeneous Graph Encoder

The graph encoder receives the same marking and propagates it over the structure of the net before classification. The graph is static and small, so message passing is implemented as two matrix products against the adjacency matrices. The encoder is a heterogeneous graph network in the R-GCN sense [21], with two node types and separate weights per relation. Those weights are written  $\phi$ , each a linear map followed by a ReLU.

The forward pass is:

$$\begin{aligned}
H_p^0 &= \phi_{\text{in}}(m) \in \mathbb{R}^{B \times |P| \times 100}, \\
H_T &= \phi_{P \rightarrow T}(A_{PT}^\top H_p^0) \in \mathbb{R}^{B \times |T| \times 100}, \\
H_p^1 &= \phi_{T \rightarrow P}(A_{TP}^\top H_T) \in \mathbb{R}^{B \times |P| \times 100}, \\
\text{enc}(m) &= \max_{p \in P} H_p^1[p] \in \mathbb{R}^{B \times 100}.
\end{aligned} \tag{4.15}$$

$A_{PT}$  records the arcs from places to transitions, with a one where place  $p$  is in the pre-set of transition  $t$ .  $A_{TP}$  records the arcs from transitions to places, with a one where place  $p$  is in the post-set of transition  $t$ . Each place starts with a single feature: its token count.  $\phi_{\text{in}}$  turns that count into a vector of 100 values, and  $H_p^0$  holds one such vector per place.  $H_T$  and  $H_p^1$  hold the same kind of vector for every transition and for every place after each product. The first product sums, for every transition, the features of the places in its pre-set. The second sums, for every place, the features of the transitions that feed it. Two hops, therefore, give each place a view of the places with which it shares a transition. The readout is a maximum because markings are sparse, and a mean would average away the few occupied places.

The adjacency matrices are registered as *buffers* and not as parameters, so they receive no gradient.  $A_{PT}$  and  $A_{TP}$  are kept as two separate matrices. Their difference is the incidence matrix, so the two matrices carry strictly more information. The two are not each other's transposes: in fig. 2.1 nothing consumes from  $p_3$ , and nothing produces into  $p_0$ .

#### 4.8.3 The Marking Sequence

The graph encoder sees the marking of the last event alone and not the history behind it. This is the *loop overwriting* of section 3.1. Table 4.5 shows it on the trace of table 4.2.

| Step | Event                  | Marking   |
|------|------------------------|-----------|
| 0    | <START> (empty prefix) | $\{p_0\}$ |
| 1    | $a$                    | $\{p_1\}$ |
| 2    | $b$                    | $\{p_2\}$ |
| 3    | $c$                    | $\{p_2\}$ |
| 4    | $d$                    | $\{p_3\}$ |

Table 4.5: Loop overwriting, on the replay of the trace of table 4.2 on the net of fig. 2.1. Markings are shown as the indices of the occupied places. After  $c$  the marking is the same as after  $b$ , so the graph encoder receives the same input twice.

The marking sequence removes the defect by encoding the whole history. The graph encoder of eq. (4.15) runs at every step, and its readouts are passed to an internal GRU,

$$R = [\text{enc}_{\text{GNN}}(m_1), \dots, \text{enc}_{\text{GNN}}(m_L)] \in \mathbb{R}^{B \times L \times 100}, \quad \text{enc}(m_{1:L}) = \text{GRU}(R)_L \in \mathbb{R}^{B \times 100}. \tag{4.16}$$

The internal GRU is packed on the same length vector as the token stream, so padding steps never enter the recurrence. An all-zero marking is *legal*, so only the lengths distinguish a padded step from a real empty state.

## 4.9 Training and Experiments

Every model is trained under the schedule of table 4.4. A minimum of five epochs is executed, then early stopping reads validation cross-entropy alone for every variant. One seed governs the initialization of the network and the draw of the corruption. Within a cell of the grid, meaning one dataset at one noise level under one seed, corruption is applied once and shared, so every variant sees the *same* corrupted log. Every grid is driven by a single script that writes one CSV row per cell. Each row records the split strategy, the knowledge source, the vocabulary scope, the three partition fractions, the training-log compliance, the accelerator it ran on, a fingerprint of the traces the knowledge is mined from, and one of the discovered net.

Chapter 5 reports the two evaluation protocols under which these models were measured.

## Chapter 5

# Experiments

This chapter reports the benchmark. It describes the logs, the two protocols, and the noise, and then answers the three research questions in order.

### 5.1 Setup

Event logs, protocols, and the training pipeline remain constant between runs. Every run is defined by the log used, the model that produced it, and the noise injected into the traces. The code, the scripts that mine the artifacts, and the result grids of both protocols are available at <https://github.com/MatteoZorzi/neuro-symbolic-ppm-thesis>.

#### 5.1.1 Data

Four publicly available event logs are used. Table 5.1 reports their size, the activity alphabet, and how much their traces vary.

| Log                              | Cases  | Events  | Activities | Length (med/max) | Variants |
|----------------------------------|--------|---------|------------|------------------|----------|
| Sepsis Cases [32]                | 1 050  | 15 214  | 16         | 13 / 185         | 846      |
| BPIC 2013 incidents [33]         | 7 554  | 65 533  | 4          | 6 / 123          | 1 511    |
| BPIC 2020 dom. declarations [34] | 10 500 | 56 437  | 17         | 5 / 24           | 99       |
| BPIC 2012 [35]                   | 13 087 | 262 200 | 24         | 11 / 175         | 4 366    |

Table 5.1: Four event logs, with total number of cases and events, distinct activities, median and maximum trace length and number of variants. Produced by `official_experiments/scripts/dataset_table.py`.

The four logs come from different domains. Sepsis is a clinical log of patients admitted with suspected sepsis. BPIC 2013 records incident management in an IT service desk. BPIC 2020 records travel-expense declarations and is close to a workflow. BPIC 2012 records a loan application process. Sepsis is shared with the reference work, and BPIC 2012 is new to the comparison. BPIC 2013 and BPIC 2020 come from the same collections, but record different processes.

### 5.1.2 Evaluation Protocols

The benchmark runs under two protocols. These protocols differ only in where the background knowledge is mined. The test protocol mines the knowledge from the test partition, following the direction of [6]. The train protocol mines the knowledge from the train partition, which is what deployed systems normally do. Table 5.2 states them side by side.

|                  | Test protocol               | Train protocol              |
|------------------|-----------------------------|-----------------------------|
| Case split       | by timestamp of first event | by timestamp of first event |
| Log filtering    | none; every case kept       | none; every case kept       |
| Vocabulary       | from all partitions         | from all partitions         |
| Knowledge source | <b>test partition</b>       | <b>training partition</b>   |
| Noise            | on every event of the trace | on every event of the trace |
| Tasks            | next activity + suffix      | next activity + suffix      |

Table 5.2: The two protocols. Partitions are 70/15/15 by case, and both are evaluated on every trained model. The suffix results come from the same network as the next-activity.

Cases are ordered by the timestamp in chronological order and cut by position, so training precedes validation, which precedes the test. The split is over cases and never over prefixes: every prefix of a case is related to every other prefix of the same case, so cutting the prefix log directly would place the same case on both sides and leak information [2]. Nothing is filtered, and the vocabulary is built over all partitions so that all activities are present. Noise replaces the activity label, and corruption propagates into every following prefix. Nine levels of noise are used:  $[0, 0.1, \dots, 0.7, 0.8]$ , the fraction of events corrupted. The reference work stops at 0.4 [6], but in this thesis, the range is doubled because that stopping point leaves the logic losses untested, where they change behavior. The baseline uses no knowledge; hence, it has the same performance for both protocols. Only what it is measured against changes. Compliance with the precedence constraints falls under noise but never reaches zero. Every level of table 5.3 leaves something for the variants to learn.

| Log                 | Test protocol |       |       | Train protocol |       |       |
|---------------------|---------------|-------|-------|----------------|-------|-------|
|                     | 0%            | 40%   | 80%   | 0%             | 40%   | 80%   |
| Sepsis Cases        | 0.969         | 0.199 | 0.037 | 0.997          | 0.217 | 0.054 |
| BPIC 2013 incidents | 1.000         | 0.726 | 0.376 | 1.000          | 0.726 | 0.376 |
| BPIC 2020 dom. dec. | 0.707         | 0.151 | 0.104 | 0.999          | 0.272 | 0.137 |
| BPIC 2012           | 1.000         | 0.245 | 0.130 | 1.000          | 0.283 | 0.135 |

Table 5.3: Fraction of training traces that still satisfy the mined constraints after noise injection, at three of the nine levels, averaged over the ten seeds. Produced by `official_experiments/scripts/compliance_table.py`.

The protocol fixes the origin of the knowledge, so it also fixes the artifacts, which table 5.4 reports. The discovered Petri net is shared by every variant, and no comparison is made between two different process models. In BPIC 2013, the net mined from the test and train sets is the same object. Therefore, they are the same experiment, and their result rows agree. This demonstrates that the pipeline is deterministic, but it means that the compari-

son between the two protocols rests on three logs and not on four.

| Log                 | Knowledge | Petri net |        |        |      | Reachability Automaton |       |         |
|---------------------|-----------|-----------|--------|--------|------|------------------------|-------|---------|
|                     |           | Places    | Trans. | Silent | Arcs | States                 | Pairs | Constr. |
| Sepsis Cases        | test      | 21        | 29     | 14     | 66   | 63                     | 140   | 15      |
|                     | train     | 27        | 35     | 20     | 82   | 220                    | 199   | 15      |
| BPIC 2013 incidents | test      | 10        | 12     | 9      | 26   | 4                      | 19    | 1       |
|                     | train     | 10        | 12     | 9      | 26   | 4                      | 19    | 1       |
| BPIC 2020 dom. dec. | test      | 18        | 25     | 15     | 50   | 11                     | 222   | 13      |
|                     | train     | 22        | 37     | 21     | 74   | 15                     | 80    | 10      |
| BPIC 2012           | test      | 45        | 56     | 36     | 124  | 32                     | 318   | 15      |
|                     | train     | 39        | 61     | 37     | 134  | 40                     | 442   | 15      |

Table 5.4: The mined artifacts, per log and per protocol. *Pairs* are the activity pairs admitted by the reachability automaton. Produced by `official_experiments/scripts/artifacts_table.py`.

## 5.2 Metrics

The benchmark measures three things: whether a prediction is correct, whether it is allowed by the process model, and how much computational time the model takes to train. A model can perform better on one and worse on another.

**Predictive Quality.** Next-activity prediction is a classification over  $\mathcal{A}$ . *Accuracy* is the measure used [2]. It is the fraction of prefixes whose highest-scoring activity is the correct one,

$$\text{acc} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\hat{a}_i = a_i], \quad (5.1)$$

with  $N$  the number of prefixes,  $a_i$  the real next activity of the  $i$ -th one, and  $\hat{a}_i = \arg \max_c p_i[c]$  its prediction.

Event logs are imbalanced because a few activities carry most of the events. A model that predicts only the frequent ones can still score well on accuracy. Accuracy is therefore never reported on its own but always alongside a conformance measure.

**Suffix Quality.** A generated suffix is compared with the real one. Comparing them position by position penalizes a prediction that is otherwise close. A single missing event shifts every later position, so a sequence that is almost correct scores as if it were wrong. The *Damerau-Levenshtein distance* avoids this problem. For two sequences  $u$  and  $v$ , it is the length of the shortest edit sequence that turns  $u$  into  $v$ . Writing  $d(i, j)$  for the distance between the first  $i$  elements of  $u$  and the first  $j$  elements of  $v$ , it is given by the recurrence

$$d(i, j) = \min \begin{cases} d(i-1, j) + 1, \\ d(i, j-1) + 1, \\ d(i-1, j-1) + \mathbb{1}[u_i \neq v_j], \\ d(i-2, j-2) + 1 \end{cases} \quad \text{if } i, j > 1, u_i = v_{j-1}, u_{i-1} = v_j, \quad (5.2)$$

with  $d(i, 0) = i$ ,  $d(0, j) = j$ , and  $DL(u, v) = d(|u|, |v|)$ , where an edit is an insertion, a deletion, a substitution, or a transposition of two adjacent elements [36, 37]. Dividing the distance by the length of the longer sequence and subtracting from one gives the similarity used on suffixes [8],

$$\text{sim}(u, v) = 1 - \frac{DL(u, v)}{\max(|u|, |v|)}, \quad (5.3)$$

which is 1 when the two sequences are identical. A short prediction cannot score well against a long suffix.

**Conformance.** Conformance is measured against the reachability automaton of the discovered Petri net. The automaton and the marking are two views of one net, as section 4.7.1 sets out. Scoring one channel against one artifact and another channel against a different one would compare methods that never received the same knowledge. A single referee is what makes the eight models comparable with one another.

The *forbidden probability mass* is the penalty term of eq. (4.6) evaluated on the test set,

$$\text{forbidden} = \frac{1}{N} \sum_{i=1}^N \sum_{c: M^{\text{net}}[t_i, c]=0} p_i[c], \quad (5.4)$$

with  $t_i$  the last token of the  $i$ -th prefix. It is the average probability that the model assigns to activities that the automaton does not permit after the last activity of the prefix.

Generated suffixes are checked with the same automaton. The *suffix violation rate* counts generated transitions. A transition is violated when the model selects an activity that the automaton does not permit after the previously generated activity. Every transition is allowed when that activity has no permitted successor.

**Computational Time.** Computational time is measured as the seconds one epoch takes. A whole run takes that time multiplied by the number of epochs. It is the only metric that measures what a channel consumes rather than what it achieves.

Every run was executed on the university HPC cluster, and every cell of the grid used the same node type and the same version of the framework. The node provides one NVIDIA A100-SXM4-80GB and AMD EPYC 7513 processor. Each job was allocated sixteen CPU cores and 64 GB of memory. The framework is PyTorch 2.13.0 built against CUDA 13.0. Times are therefore comparable across models and across logs.

## 5.3 Results

This section presents the results, one metric at a time. Section 5.3.1 stages every model against the baseline on all four metrics at once. The subsections that follow analyze each metric, separating the four logs. The research questions are answered later in section 5.4.

### 5.3.1 Overview

5760 runs is the number of records run between eight models, four logs, nine levels of noise, ten seeds, and two protocols. The benchmark cannot be fitted into this document. A reduction of the benchmark is provided inside Table 5.5 so that nothing useful to the research questions is lost.

| Knowl. | Model<br><i>better if</i> | Accuracy<br>↑ + |       | Forbidden mass<br>↓ − |       | DL similarity<br>↑ + |       | Suffix violations<br>↓ − |       |
|--------|---------------------------|-----------------|-------|-----------------------|-------|----------------------|-------|--------------------------|-------|
| test   | Local loss                | −1.3%           | 13/36 | −99.0%                | 36/36 | −3.9%                | 9/36  | −95.1%                   | 36/36 |
|        | Global loss               | +1.6%           | 24/36 | −32.2%                | 36/36 | +1.3%                | 21/36 | −88.2%                   | 35/36 |
|        | Projected net             | +1.2%           | 22/36 | −23.9%                | 36/36 | +0.5%                | 25/36 | −19.3%                   | 28/36 |
|        | State-indexed net         | +1.2%           | 27/36 | −11.2%                | 36/36 | +1.7%                | 25/36 | −7.3%                    | 25/36 |
|        | Marking                   | +0.1%           | 18/36 | +0.6%                 | 15/36 | −0.1%                | 18/36 | +14.0%                   | 19/36 |
|        | Graph encoder             | +0.1%           | 19/36 | −0.1%                 | 17/36 | +0.1%                | 24/36 | +6.7%                    | 17/36 |
|        | Marking sequence          | −0.3%           | 14/36 | −0.3%                 | 23/36 | +0.0%                | 18/36 | −5.7%                    | 23/36 |
| train  | Local loss                | +0.8%           | 12/36 | −98.9%                | 36/36 | −1.5%                | 12/36 | −92.2%                   | 36/36 |
|        | Global loss               | +1.5%           | 23/36 | −34.0%                | 36/36 | +1.1%                | 18/36 | −98.7%                   | 36/36 |
|        | Projected net             | +1.2%           | 25/36 | −23.8%                | 36/36 | +0.7%                | 23/36 | −18.0%                   | 29/36 |
|        | State-indexed net         | +1.2%           | 25/36 | −15.0%                | 36/36 | +1.7%                | 26/36 | −3.6%                    | 29/36 |
|        | Marking                   | +0.0%           | 21/36 | +0.7%                 | 14/36 | +0.1%                | 15/36 | +12.0%                   | 23/36 |
|        | Graph encoder             | +0.1%           | 22/36 | −0.1%                 | 20/36 | +0.1%                | 26/36 | +3.1%                    | 21/36 |
|        | Marking sequence          | −0.1%           | 16/36 | −0.6%                 | 26/36 | +0.1%                | 19/36 | −6.8%                    | 26/36 |

Table 5.5: Every model goes against the baseline. The percentage is the mean difference, while the fraction counts the cells in which the model is better or equal to the baseline. Produced by `official_experiments/scripts/summary_table.py`.

A cell is one log at one level of noise. The value of each cell is the median of the ten seeds. Each model is then paired against the baseline of the same cell. Four logs and nine levels give thirty-six cells per protocol. The percentage is the mean of those differences, relative to the baseline mean over the same cells.

Performance differences can be isolated from run-to-run variation using paired comparisons on identical data. Three models stand out. The global loss, the projected net and the state-indexed net have every sign in the desired direction under both protocols. The margins on accuracy and similarity are small; none of the three predicts better, and the global loss is the one that allows for the most conformance. The local loss is the opposite. It allows for the largest reduction of forbidden mass in exchange for lower performance.

Singular medians are not comparable across logs; the values are different, but the percentages are not. A complete analysis of each metric follows. The figures are produced by `official_experiments/scripts/metric_figures.py`, and every point in them is the median over the ten seeds.

Noise is injected only in the training partition, therefore, the validation and test sets remain clean. Every variant inside a cell works with the same corrupted log, and this implies that the difference in results cannot come from the data.

Given a seed, the baseline produces a set of initial weights and a batch order. The four losses leave the architecture intact; hence, they start from the same values, and each of them differs from the baseline in the objective alone.

The net mined from BPIC 2013 carries the same fingerprint in both the test and training partitions. The two protocols are shown to coincide there.

Every reported accuracy and suffix generation represents the raw model output. No mask is applied; otherwise, the violation rate would be zero by construction.

### 5.3.2 Next-Activity Accuracy

Next-activity accuracy is the fraction of prefixes whose highest-scoring activity is the correct one. Figure 5.1 reports it against noise, one row per log and one column per protocol. It is what any knowledge channel must leave intact to be worth using.

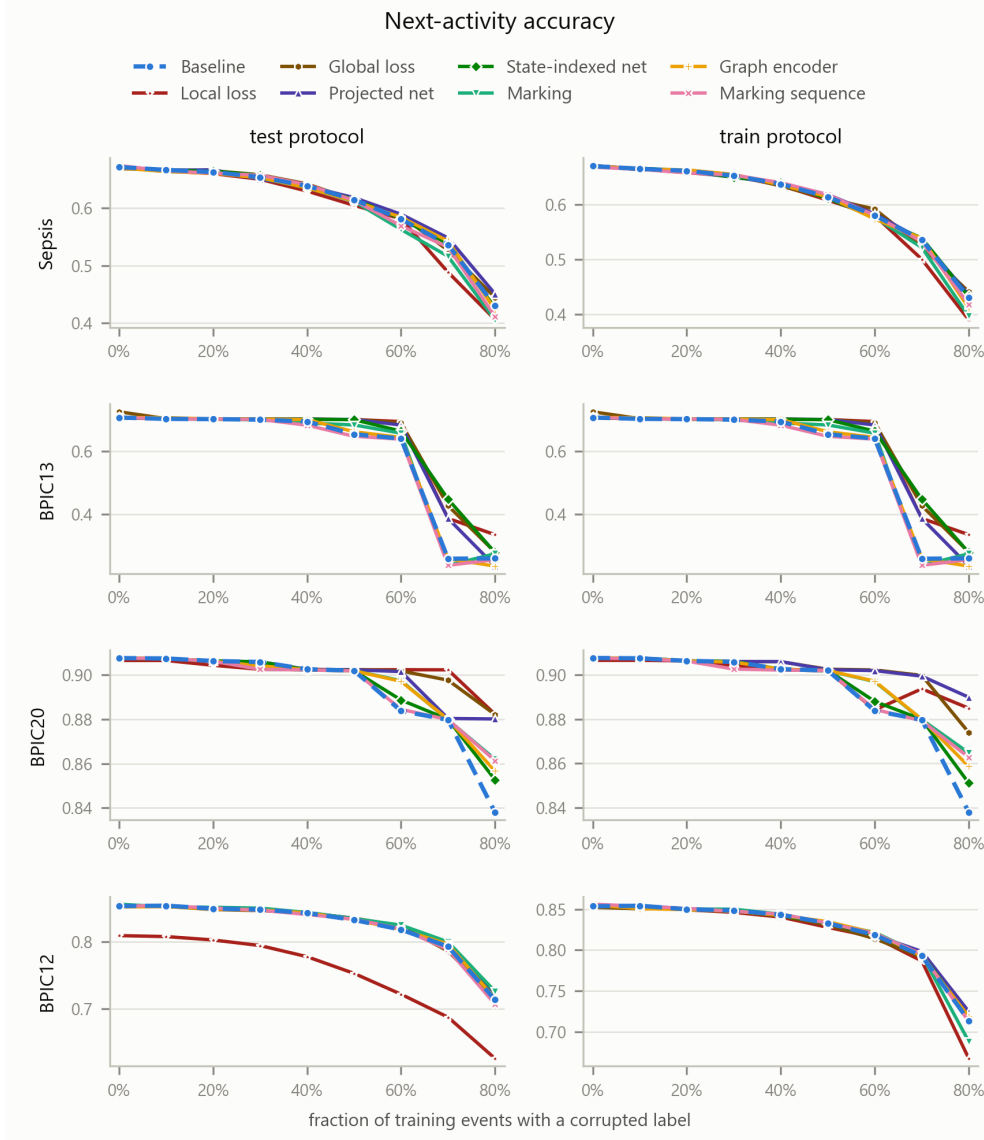


Figure 5.1: Next-activity accuracy under both protocols.

There is no difference in accuracy between the eight models. The distance between the best and the worst is 0.012 at the median. An exception exists: the local loss on BPIC 2012, which results in a loss of 0.070 in accuracy under the test protocol and 0.008 under the training protocol. Past 60% of noise, BPIC 2013 collapses: the baseline falls from 0.64 to 0.26, and at 70% the four losses hold between 0.39 and 0.45 before falling with it. No channel can raise accuracy by a consistent margin across the four logs.

### 5.3.3 Suffix Violation Rate

The suffix violation rate is the fraction of the steps of a generated suffix that the automaton does not permit after the previously generated activity. Figure 5.2 reports it against noise, one row per log and one column per protocol, and it is where an error compounds because the model generates from its own predictions.

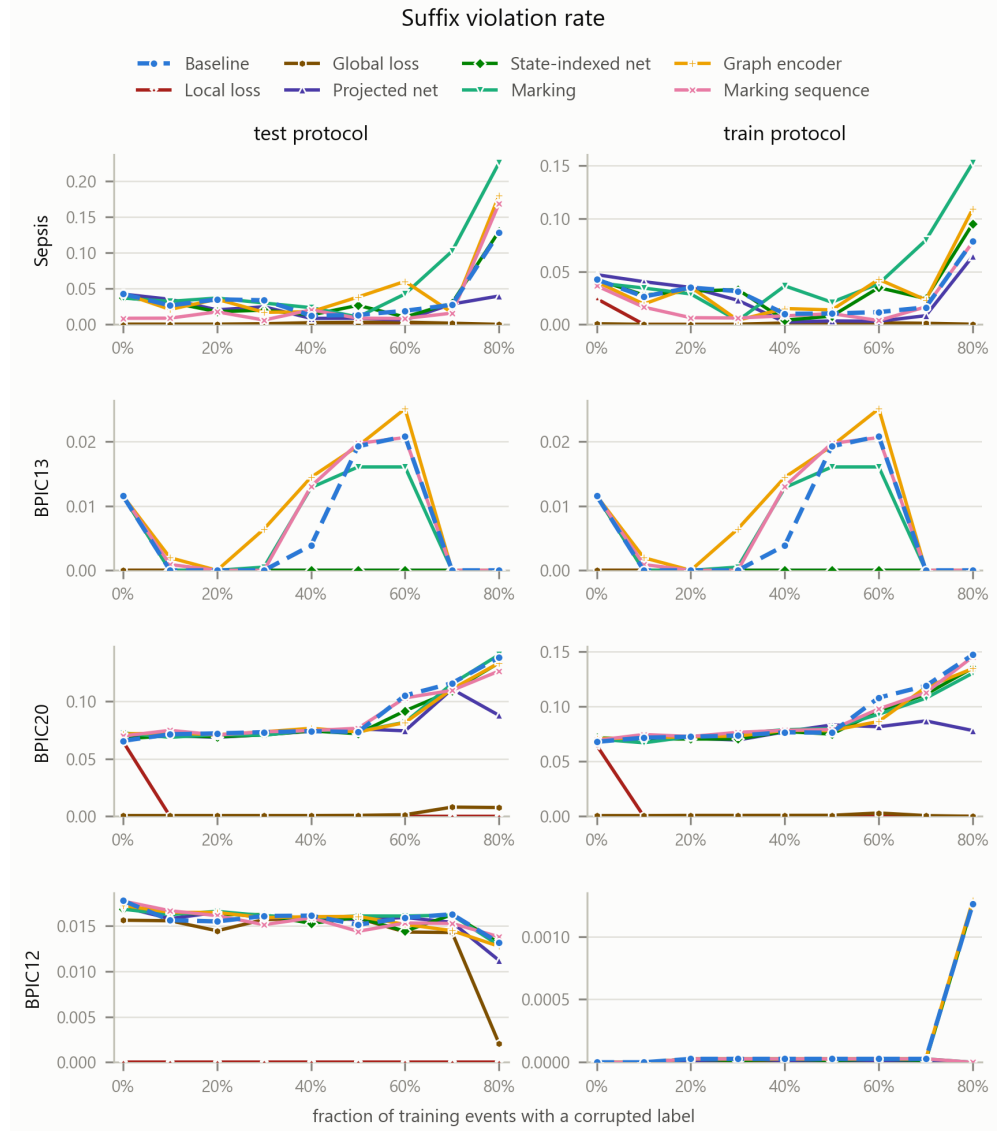


Figure 5.2: Suffix violation rate under both protocols.

The state of the art losses have an almost zero rate of suffix violation. The local loss is exactly zero in 96% of the cells, while the global loss never exceeds 0.016. The other six do not move together: their order changes at almost every level. On BPIC 13, every model is at zero past 60% noise, and on BPIC 12, none exceeds 0.0013 under the train protocol.

### 5.3.4 Suffix Similarity

Suffix similarity is the Damerau-Levenshtein similarity of eq. (5.3) between the generated suffix and the real one. Figure 5.3 reports it against noise, one row per log and one column per protocol, and it is what a model conceding conformance has to give up in exchange.

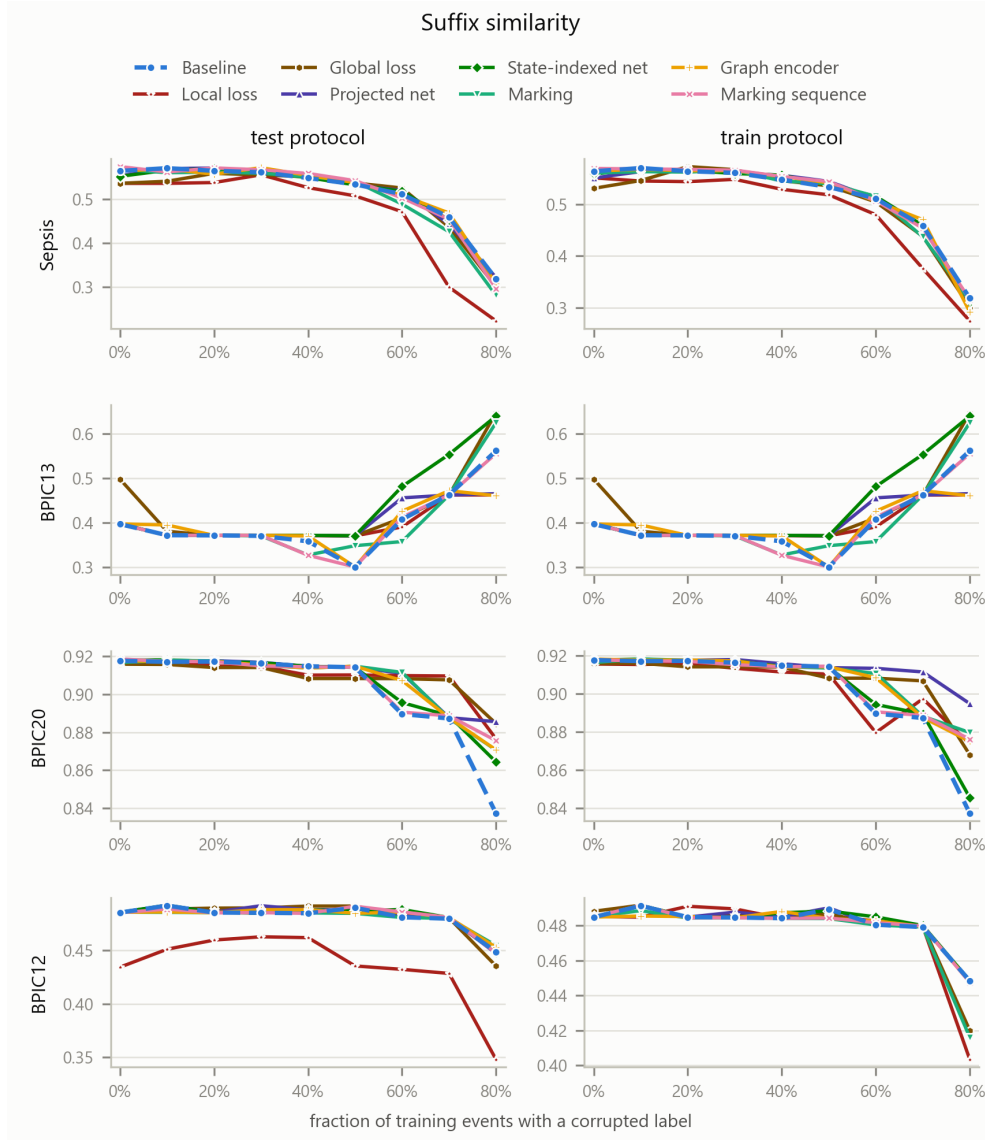


Figure 5.3: Suffix similarity under both protocols.

The eight models are very close to each other: the median distance between the best and the worst is 0.029. The local loss is the only model that earns in conformance: in the test protocol, it loses 0.048 on Sepsis and 0.046 on BPIC 2012. At 80% noise on Sepsis, the similarity falls to 0.224 against the baseline's 0.319. Under the train protocol, the cost falls to 0.029 on Sepsis and nearly vanishes on BPIC 2012. BPIC 2013 goes against expectations: the baseline ends at 0.561 after starting at 0.397. The log has four activities, so a suffix allows for every possible combination. The rise is more likely a side effect of that alphabet than a gain in prediction.

### 5.3.5 Forbidden Probability Mass

Forbidden probability mass is the probability that the model places on activities that the automaton does not permit after the last activity of the prefix. Figure 5.4 reports it against noise, one row per log and one column per protocol, and it is the quantity that the local loss and the projected net directly penalize.

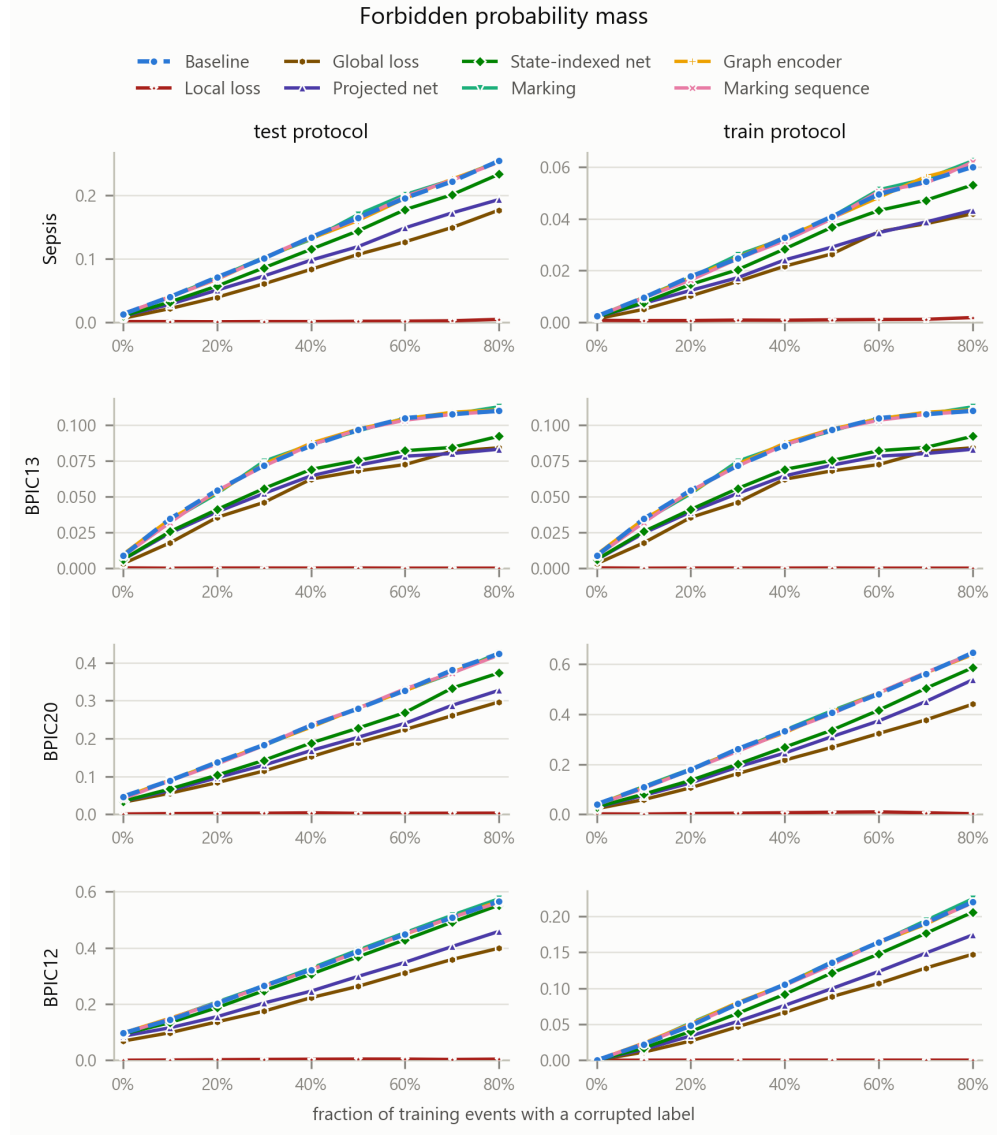


Figure 5.4: Forbidden probability mass under both protocols.

Noise increases the forbidden probability mass with every percentage increase. The local loss is flat, has less mass than any other model in every cell but one, and it does not increase above 0.010. Excluding the local loss, the other three losses scale the curve down, almost always in the same order: the global loss is the lowest, and the state-indexed net is the highest. On this metric, only the loss channel moves the curve: the three feature models stay on the baseline, and the form that knowledge takes only decides how far the curve drops.

Figure 5.5 ranks the eight models inside every run, one log at one noise level under one seed, and compares their average ranks with the Friedman test and the Nemenyi critical difference [38]. The local loss ranks first under both protocols, and each of the four losses is separated from every other model. The baseline and the three feature models form a single group that the test cannot separate.

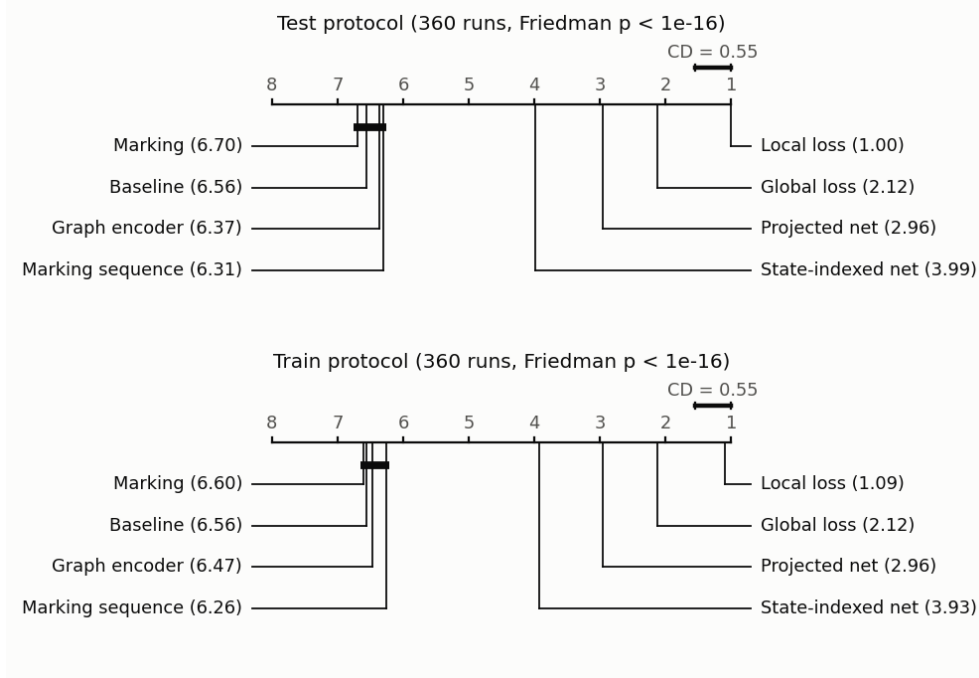


Figure 5.5: Critical difference diagram of the forbidden probability mass under both protocols. The axis reports the average rank over the 360 runs of each protocol, lower is better, and models joined by a bar are not significantly different at the 0.05 level. Produced by `official_experiments/scripts/cd_diagram.py`.

### 5.3.6 Computational Time

Computational time is the seconds one epoch takes, and the noise does not affect it: inside a cell, the nine levels spread without direction. Table 5.6 reports that time, and fig. 5.6 sets it against what it buys.

An epoch is 1.3 seconds on Sepsis and 21.2 on BPIC 2012. The ratios are taken from inside the log. The marking sequence requests the most on that log, with 3.3 times the baseline. This is because it carries one marking per event. The last column indicates the median number of epochs: the global loss runs thirteen epochs against twelve for the baseline and nine for the local loss.

| Model             | Seconds of one epoch |        |        |        | Seconds of one run |        |        |        | × baseline |      |        |
|-------------------|----------------------|--------|--------|--------|--------------------|--------|--------|--------|------------|------|--------|
|                   | Sepsis               | BPIC13 | BPIC20 | BPIC12 | Sepsis             | BPIC13 | BPIC20 | BPIC12 | epoch      | run  | Epochs |
| Baseline          | 1.34                 | 4.36   | 1.93   | 21.18  | 16                 | 52     | 23     | 254    | 1.00       | 1.00 | 12.0   |
| Local loss        | 1.27                 | 4.58   | 2.06   | 20.76  | 11                 | 41     | 19     | 187    | 1.02       | 1.01 | 9.0    |
| Global loss       | 3.02                 | 8.68   | 4.58   | 47.82  | 38                 | 111    | 58     | 609    | 2.26       | 2.53 | 12.8   |
| Projected net     | 1.20                 | 4.36   | 1.96   | 20.70  | 15                 | 54     | 24     | 259    | 1.00       | 0.99 | 12.5   |
| State-indexed net | 1.21                 | 4.42   | 1.98   | 21.20  | 16                 | 57     | 26     | 275    | 1.00       | 1.02 | 13.0   |
| Marking           | 1.41                 | 4.64   | 2.13   | 22.04  | 18                 | 58     | 27     | 276    | 1.08       | 1.10 | 12.5   |
| Graph encoder     | 1.40                 | 5.28   | 2.69   | 21.77  | 18                 | 67     | 34     | 278    | 1.14       | 1.16 | 12.8   |
| Marking sequence  | 3.22                 | 8.44   | 4.28   | 69.51  | 37                 | 97     | 49     | 800    | 2.24       | 2.14 | 11.5   |

Table 5.6: Time of one epoch in seconds, the seconds of one run as that time multiplied by the epochs a run goes through, that time and a whole run against the baseline of the same log, and the epochs a run goes through. Medians over seeds, noise and protocol. Produced by `official_experiments/scripts/cost_table.py`.

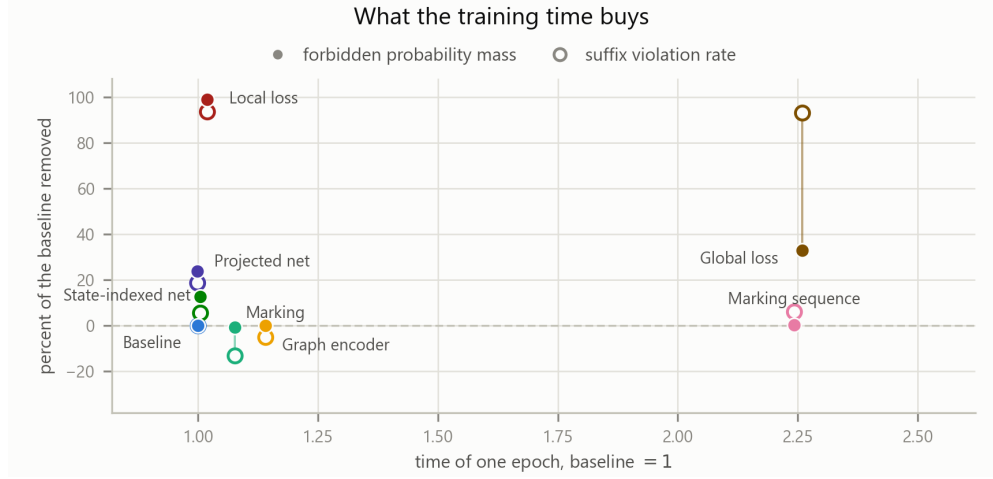


Figure 5.6: Each model is one point: what an epoch costs in time against how much of the baseline’s non-conformance it removes. Medians over seeds, noise and protocol. Produced by `official_experiments/scripts/cost_figure.py`.

Six models train in the same time as the baseline, and two exceed it. For the same price, the local loss removes more than 90% of the forbidden mass and suffix violations. The other two train in twice the time: the global loss reduces over 93% of the violations and a third of the mass; the marking sequence does not.

Figure 5.7 applies the same test to the seconds of one run. The global loss and the marking sequence are the slowest under both protocols, and the test separates them from every other model. Among the other six, the local loss, the projected net, and the baseline cannot be separated. The seconds of a run include the epochs before early stopping, so the local loss is among the fastest also because it stops earlier.

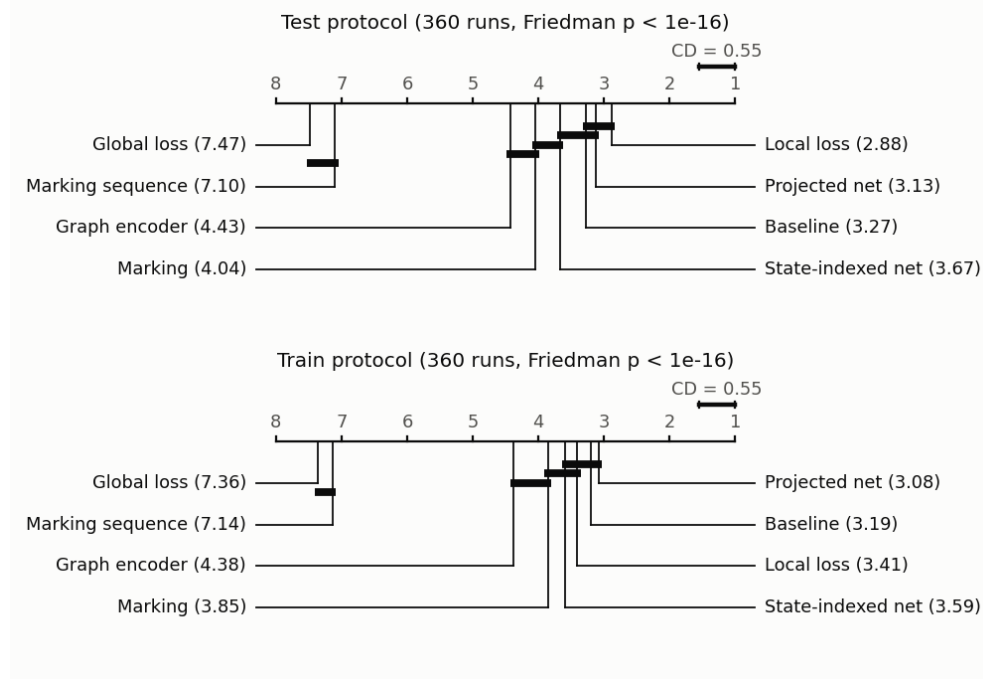


Figure 5.7: Critical difference diagram of the seconds of one run under both protocols. The axis reports the average rank over the 360 runs of each protocol, lower is faster, and models joined by a bar are not significantly different at the 0.05 level. Produced by `official_experiments/scripts/cd_diagram.py`.

## 5.4 Answers to the Research Questions

This section answers the three questions of section 1.3. Each answer reviews the results of section 5.3 and indicates what they support. The two protocols agree on all three: the answers do not depend on the partition from which the knowledge is mined.

### 5.4.1 RQ1: The Same Knowledge as a Feature Achieves None of It

The loss models extract the same automaton from the Petri net and add a penalty to the cross entropy. The three feature models, instead, read the same Petri net, and each elaborates on the extracted marking one step further. First, the marking is reached by token replay; then, that marking is passed over the bipartite graph of the net, and finally, the sequence of markings along the prefix is read. None of them obtains any part of the effect.

On forbidden probability mass, their curves follow the baseline in fig. 5.4, and every difference in table 5.5 stays within one percent of it. On the generated suffix, the marking and the graph encoder are worse than the baseline under the test protocol, by fourteen and seven percent.

The two channels do not inject different knowledge, since a marking is a deterministic function of the same information that the automaton encodes. The marking is concatenated to the final state of the LSTM, so it enters as evidence and competes with everything else that the recurrence has learned, while a loss acts on the quantity being scored. The channel decides whether the knowledge is used at all.

#### 5.4.2 RQ2: The Form of the Automaton Matters, Its Memory Does Not

In the loss channel, both variants derive different knowledge from the same automaton. The projection projects it onto directly-follows pairs, so the constraint is indexed by the last activity. The state-indexed form keeps the states, so the constraint can express dependencies of unbounded depth. The second is more informative, but the first works better on conformance. Under the test protocol, the projection removes twice the forbidden mass of the state-indexed form and two and a half times the suffix violations. Accuracy is the same under both protocols, and on similarity, the state-indexed form is ahead by a point. Keeping the states costs something. The same training signal is split over many more indices, so each one sees fewer examples, and the constraint is weaker. The measurement also favors the projection because both conformance metrics are indexed by the last activity, which is the mask on which the projection trains.

The order by forbidden mass reduction is the local loss, the global loss, the projection, and the state-indexed form, under both protocols. This order represents how hard each penalty pushes on the measured quantity. It is not the order of expressive power, nor of how much of the net each variant reads. The local loss and the projection read the same mask, but the local loss penalizes  $-\log(1 - r)$ , which grows without bound as the forbidden mass approaches one. The global loss never penalizes that mask, and it still removes a third of it.

#### 5.4.3 RQ3: What Works Is Also What Takes Least Time

Each channel takes a different computational time to train, and the answer of table 5.6 splits the eight models into two groups. Only the global loss and the marking sequence take longer, while the rest remains near the baseline. The local loss removes almost everything at the baseline's computational time, but beyond sixty percent noise, it loses similarity on three logs and accuracy on two. The two net-derived losses remove, respectively, a quarter and a ninth of the forbidden mass, and take the same time as the baseline. The global loss costs twice as much and reduces a third of the mass, which places it second in that metric, although it reaches the suffix nearly as well as the local loss.

The elaboration of the encoding in the feature channel increases the computational time without producing any gain with respect to what the baseline does. The marking sequence is the worst: it takes the same time as the global loss and produces nothing.

The symbolic part is paid once per log. It is negligible on three logs under both protocols, but on BPIC 2012, the reachability automaton costs 161 seconds under the train protocol. The two channels also differ after training: the loss is used only to fit the model, while a feature needs to be computed for every prefix at prediction time. A channel that leaves conformance and accuracy unchanged at the baseline's computational time can be defended as an inductive bias, while one that doubles that time cannot.

## Chapter 6

# Conclusion

This thesis asks whether the same process knowledge works through more than one channel, whether the form of the process model specification matters among the losses, and how much computational time each channel costs. The channel decides whether the knowledge influences the predictions or not. Computational time separates the methods in the opposite direction from the expected one.

### 6.1 Answers to the Research Questions

**RQ1: The same knowledge as a feature achieves nothing.** The marking, the marking on the net's graph, and the sequence of markings are derived from the same discovered Petri net that the losses read. None of them influences forbidden probability mass by more than one percent of the baseline. State, structure, and history were added one at a time, so the flat outcome cannot be blamed on missing information. The difference between the channels is not the knowledge, but rather where the knowledge is used.

**RQ2: The form of the automaton matters, its memory does not.** Both automaton-derived penalties, the projection and the state-indexed one, work; on conformance, the first works better even though it is simpler. The projection onto activity pairs removes about a quarter of the forbidden mass under both protocols. The state-indexed form removes a ninth under test and a seventh under train. Indexing by state spreads the same training signal over many more indices. Each constraint is then estimated from fewer examples, and the conformance it buys decreases. The expectation was the opposite, and it was wrong.

**RQ3: What works is also what takes least time.** The local loss trains at the price of the baseline and is the most effective variant in the benchmark. Past sixty percent corruption reduces similarity on three logs and accuracy on two, and the reference range of up to forty percent could not have exposed that. The global loss takes twice as much time and removes a third of the forbidden mass, whereas the local loss removes almost all of it. The global loss gains similarity and accuracy at the same time. Inside the feature channel, the computational time rises with the elaboration of the encoding, and despite the complexity, the gains remain zero. The marking sequence takes the same time as the global loss and still gains nothing. A channel that leaves conformance and accuracy unchanged at the baseline's computational time is a harmless inductive bias. One that leaves them unchanged at twice that time is not.

## 6.2 Contribution of the Two Protocols

The two protocols support all three answers and differ only in where the knowledge is mined: the test partition or the training partition. The answer to each question is the same under both. The reference work mines from the test partition, and this benchmark shows that the choice does not change the answers. A single protocol could not have shown it.

## 6.3 Limitations

The evidence consists of four event logs, and two of the four are short-trace administrative processes. Every constraint is mined, and none is specified. A written Declare model states what a process should do, while a net discovered by a miner states what a log did. The results indicate the value of a mined net and not the value of a written model.

The conformance metrics are indexed by the last activity, which is what the local loss and the projection penalize. Their reductions are partly true by construction. Three results are not. The global loss never penalizes that mask but reduces it anyway and gains similarity. The feature channel reads the same net and obtains nothing. The penalties do not rank in the order of their expressive power.

Every generated suffix stops at the true length because no model here has an end-of-trace symbol. The similarity values are therefore valid within this benchmark.

## 6.4 Future Work

The conformance metrics are derived from the same object that the penalties optimize. Scoring every variant against an automaton mined from the validation partition would remove that advantage. Splitting the suffix metrics by suffix length would show how much of the suffix result is generated by several steps. Neither change requires retraining; both are a second pass over the saved checkpoints. Giving the models an end-of-trace token costs more but would make the suffix results comparable to the literature. It would also expose other aspects of generation, such as whether the injected knowledge helps a model decide when a trace ends.

# Bibliography

- [1] Wil M. P. van der Aalst et al. Process mining manifesto. In *Business Process Management Workshops (BPM 2011)*, pages 169–194, 2012.
- [2] Chiara Di Francescomarino, Ivan Donadello, and Fabrizio Maria Maggi. *Predictive Process Monitoring*. Springer, Cham, 2026.
- [3] Fabrizio Maria Maggi, Chiara Di Francescomarino, Marlon Dumas, and Chiara Ghidini. Predictive monitoring of business processes. In *Advanced Information Systems Engineering (CAiSE 2014)*, pages 457–472, 2014.
- [4] Maja Pesic, Helen Schonenberg, and Wil M. P. van der Aalst. DECLARE: Full support for loosely-structured processes. In *11th IEEE International Enterprise Distributed Object Computing Conference (EDOC 2007)*, pages 287–298, 2007.
- [5] Tadao Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541–580, 1989.
- [6] Axel Mezini. Neuro-symbolic predictive process monitoring. Master’s thesis, Free University of Bozen-Bolzano, Bolzano, Italy, 2025. Extended version of [7].
- [7] Axel Mezini, Elena Umili, Ivan Donadello, Fabrizio Maria Maggi, Matteo Mancanelli, and Fabio Patrizi. Neuro-symbolic predictive process monitoring. *Information Systems*, 141:102762, 2026. Condensed version of [6].
- [8] Niek Tax, Ilya Verenich, Marcello La Rosa, and Marlon Dumas. Predictive business process monitoring with LSTM neural networks. In *Advanced Information Systems Engineering (CAiSE 2017)*, pages 477–492, 2017.
- [9] Julian Theis and Houshang Darabi. Decay replay mining to predict next process events. *IEEE Access*, 7:119787–119803, 2019.
- [10] Efrén Rama-Maneiro, Juan C. Vidal, and Manuel Lama. Embedding graph convolutional networks in recurrent neural networks for predictive monitoring. *IEEE Transactions on Knowledge and Data Engineering*, 36(1):137–151, 2024.
- [11] Ziwei Xu, Yogesh S. Rawat, Yongkang Wong, Mohan S. Kankanhalli, and Mubarak Shah. Don’t pour cereal into coffee: Differentiable temporal logic for temporal action segmentation. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 14890–14903, 2022.
- [12] Yaqi Xie, Fan Zhou, and Harold Soh. Embedding symbolic temporal knowledge into deep sequential models, 2021.

- [13] Chiara Di Francescomarino and Chiara Ghidini. Predictive process monitoring. In *Process Mining Handbook*, pages 320–346. Springer, 2022.
- [14] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI 2013)*, pages 854–860, 2013.
- [15] Wolfgang Reisig. *Understanding Petri Nets: Modeling Techniques, Analysis Methods, Case Studies*. Springer, Berlin, Heidelberg, 2013.
- [16] Sander J. J. Leemans, Dirk Fahland, and Wil M. P. van der Aalst. Discovering block-structured process models from event logs containing infrequent behaviour. In *Business Process Management Workshops (BPM 2013)*, pages 66–78, 2014.
- [17] Michael O. Rabin and Dana Scott. Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2):114–125, 1959.
- [18] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [19] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, pages 1263–1272, 2017.
- [20] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *5th International Conference on Learning Representations (ICLR 2017)*, 2017.
- [21] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web (ESWC 2018)*, pages 593–607, 2018.
- [22] Kota Sueyoshi and Takashi Matsubara. Predicated diffusion: Predicate logic-based attention guidance for text-to-image diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8651–8660, 2024.
- [23] Elena Umili and Roberto Capobianco. Deepdfa: Automata learning through neural probabilistic relaxations, 2024.
- [24] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax, 2017.
- [25] Sebastiano Dissegna and Chiara Di Francescomarino. Graph neural networks for PPM: Review and benchmark for next activity predictions. In *Business Process Management Workshops*, pages 31–43, 2025.
- [26] Andrea Chiorrini, Claudia Diamantini, Laura Genga, and Domenico Potena. Multi-perspective enriched instance graphs for next activity prediction through graph neural network. *Journal of Intelligent Information Systems*, 61(1):5–25, 2023.
- [27] Ivan Donadello, Paolo Felli, Craig Innes, Fabrizio Maria Maggi, and Marco Montali. LTL-based conformance checking of fuzzy event logs. *Process Science*, 2(1):14, 2025.

- [28] Alessandro Berti, Sebastiaan J. van Zelst, and Wil M. P. van der Aalst. Process mining for Python (PM4Py): Bridging the gap between process- and data science. In *Proceedings of the ICPM Demo Track 2019, co-located with the 1st International Conference on Process Mining (ICPM 2019)*, pages 13–16, 2019.
- [29] Fabrizio Maria Maggi, R. P. Jagadeesh Chandra Bose, and Wil M. P. van der Aalst. Efficient discovery of understandable declarative process models from event logs. In *Advanced Information Systems Engineering (CAiSE 2012)*, pages 270–285, 2012.
- [30] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [31] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- [32] Felix Mannhardt. Sepsis cases – event log. Eindhoven University of Technology, 2016.
- [33] Ward Steeman. BPI challenge 2013, incidents. Ghent University, 2013.
- [34] Boudewijn van Dongen. BPI challenge 2020: Domestic declarations. 4TU.Centre for Research Data, 2020.
- [35] Boudewijn van Dongen. BPI challenge 2012. Eindhoven University of Technology, 2012.
- [36] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.
- [37] Fred J. Damerau. A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3):171–176, 1964.
- [38] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

*Declaration on the use of AI:* Artificial intelligence tools were used exclusively as support for the linguistic and formal revision of the text. All contents, as well as the analysis, interpretation of the results, and conclusions, were formulated solely by the author.